

Infron 与 OpenRouter 路由、缓存、成本与流式性能 A/B 实验报告

摘要与结论大纲

关键词: Prompt Caching; A/B Testing; Provider Routing; Cache Affinity; Latency; Throughput; Cost Attribution; minimax/minimax-m2.7

摘要

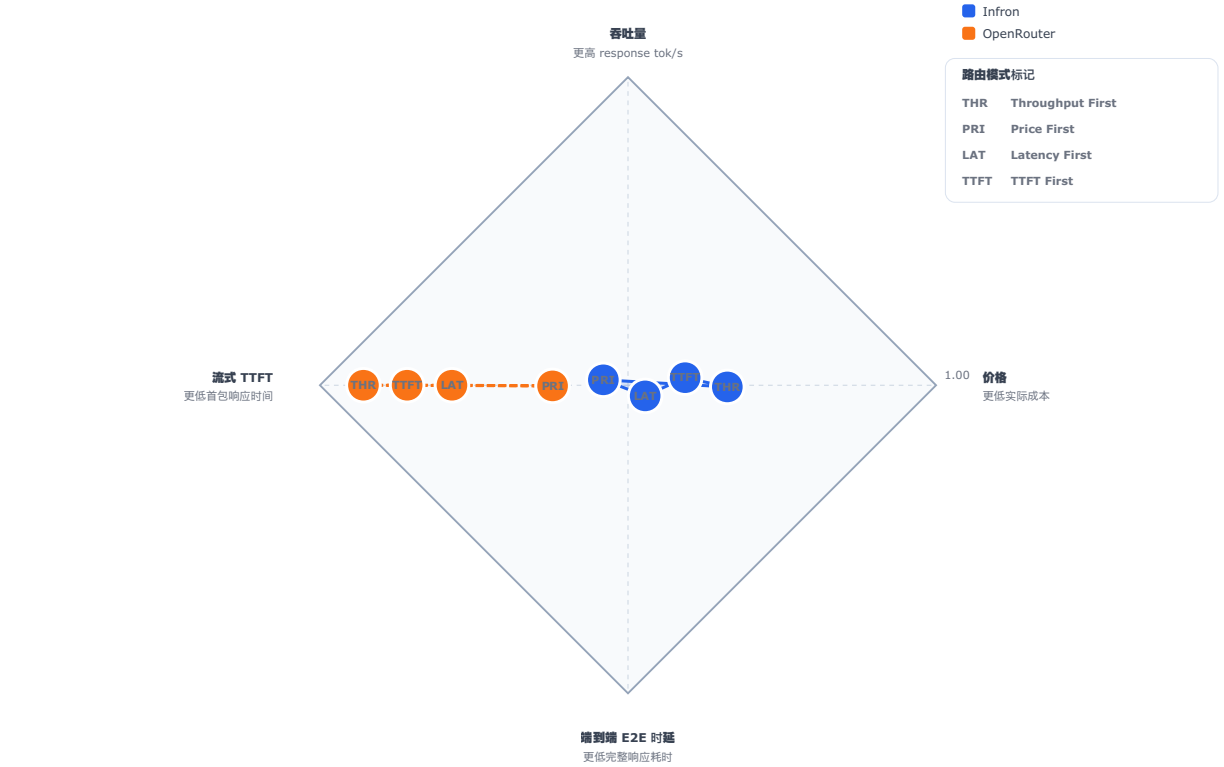
本报告以 minimax/minimax-m2.7 为对象, 对比 Infron 与 OpenRouter 在 Prompt Caching 场景下的路由策略、缓存命中、实际成本、吞吐量、TTFT 首包响应时间和端到端时延。实验包含 4 个实验组、每组 50 轮, 覆盖 4 种 routing sort 策略; 其中 TTFT First 中, Infron 使用 provider.sort=ttft, OpenRouter 使用其支持的 provider.sort=latency 作为对照。经过异常 usage、HTTP 异常和 A/B input tokens 偏差超过 50 tokens 的样本剔除后, 最终保留 800 个配对样本、3200 次请求级观测记录, 剔除 0 条记录。

核心结论是: 在 usage.prompt_tokens 偏差不超过 50 tokens 的配对样本中, Infron 在 throughput 和 price 模式下, Token 级缓存命中率更高, OpenRouter 在 latency 和 ttft 模式下, Token 级缓存命中率更高; Infron 在所有路由模式下, 实际成本都更低; Infron 在 price 模式下, 吞吐量更高, OpenRouter 在 throughput、latency 和 ttft 模式下, 吞吐量更高; Infron 在 price 模式下, 时延更低, OpenRouter 在 throughput、latency 和 ttft 模式下, 时延更低; Infron 在 price 模式下, TTFT 首包响应时间更低, OpenRouter 在 throughput、latency 和 ttft 模式下, TTFT 首包响应时间更低。整体看, Infron 的优势集中在成本控制、部分模式的低时延路径, OpenRouter 的优势集中在吞吐、TTFT、部分模式的端到端时延表现。平台选择应围绕业务目标展开, 单一指标不足以代表整体效果。

本轮未显式控制 reasoning/thinking, 响应中的 reasoning tokens 作为观测变量记录。

Inference 平台“不可能四角”：四项核心指标的严格归一化对比

单图投影展示：每个路由模式先做四项指标 A/B 归一化，再投影成一个点；同一平台的四个点连接成区域。



读图：THR/PRI/LAT/TTFT 分别代表四种路由模式；蓝色区域为 Infron，橙色区域为 OpenRouter，区域外扩方向表示对应指标优势方向。
该图采用统一归一化，四维到二维投影和一致径向视觉放大，用于总览区域形状；精确逐指标胜负以下方表格为准。

图 0：Inference 平台“不可能四角”。吞吐量、价格、端到端时延和 TTFT 很难同时达到最优，平台路由通常会在四个方向之间做取舍；图中将四项归一化指标投影为路由模式点，并将同一平台的四个点连接成区域。

结论总览：核心指标与路由模式胜出方

基于严格 A/B 配对样本；每个单元格显示同一 routing sort 下表现更好的一方。

缓存命中率 Infron 2/4 胜 最大优势 48.10% 越高越好	实际成本 Infron 4/4 胜 最大优势 89.73% 越低越好	吞吐量 OpenRouter 3/4 胜 最大优势 26.62% 越高越好	时延 OpenRouter 3/4 胜 最大优势 21.02% 越低越好	TTFT OpenRouter 3/4 胜 最大优势 23.55% 越低越好
--	---	--	---	---

路由模式	吞吐达成	成本达成	时延达成	TTFT 达成	缓存命中	路由模式达成情况
Throughput First	OpenRouter 目标 高 26.62%	Infron 低 89.73%	OpenRouter 低 21.02%	OpenRouter 低 23.55%	Infron 高 4.14%	吞吐目标: OpenRouter 高 26.62%
Price First	Infron 高 45.15%	Infron 目标 低 51.73%	Infron 低 31.13%	Infron 低 29.89%	Infron 高 48.10%	成本目标: Infron 低 51.73%
Latency First	OpenRouter 高 3.47%	Infron 低 88.82%	OpenRouter 目标 低 3.34%	OpenRouter 低 9.63%	OpenRouter 高 0.48%	时延目标: OpenRouter 低 3.34%
TTFT First	OpenRouter 高 14.13%	Infron 低 89.71%	OpenRouter 低 12.37%	OpenRouter 目标 低 18.49%	OpenRouter 高 0.46%	TTFT 目标: OpenRouter 低 18.49%

读法：前四列与四种路由模式顺序对齐；金色对角线表示该路由模式目标指标的胜出方。
缓存和吞吐越高越好，成本、时延和 TTFT 越低越好；缓存命中作为跨模式辅助指标单独放在最后一列。

Infron
OpenRouter

图 A：结论总览图。上方卡片概括跨路由模式的总体胜出方，下方矩阵按 throughput、price、latency、TTFT 的路由目标顺序组织列，金色对角线表示各路由模式目标指标的 A/B 胜出方。

结论大纲

研究维度	结论	证据位置
控制变量	进入统计的 A/B 样本满足同一 sort/group/round 下 first/second 请求 usage.prompt_tokens 各自偏差不超过 50 tokens；各模式 Input Tokens 对照为 throughput =6207102/6207808； price =6207102/6207808； latency =6207008/6207808； ttft =6207008/6207808	方法与数据质量章节
缓存复用	Infron 在 throughput 和 price 模式下，Token 级缓存命中率更高，OpenRouter 在 latency 和 ttft 模式下，Token 级缓存命中率更高，说明本轮 provider stick/cache affinity 已转化为可观测的缓存命中差异	结果与机制分析章节
实际成本	Infron 在所有路由模式下，实际成本都更低，成本差异与 cache read tokens 同向变化	结果与结论章节
性能表现	Infron 在 price 模式下，吞吐量更高，OpenRouter 在 throughput、latency 和 ttft 模式下，吞吐量更高；Infron 在 price 模式下，时延更低，OpenRouter 在 throughput、latency 和 ttft 模式下，时延更低；Infron 在 price 模式下，TTFT 首包响应时间更低，OpenRouter 在 throughput、latency 和 ttft 模式下，TTFT 首包响应时间更低	结果可视化与结论章节
Reasoning 控制	本轮未显式控制 reasoning/thinking，响应中的 reasoning tokens 作为观测变量记录。	Reasoning / Thinking 控制校验章节
归因边界	报告只使用响应可观测 telemetry，包括 provider 字段、usage、cost breakdown、TTFT、latency 和 cache tokens；未把平台内部私有 routing trace 当作已观测事实	机制分析、下钻分析与局限性章节
业务含义	对稳定长上下文、RAG 前缀、Agent 工具说明和批处理任务，缓存命中率与成本可预测性是核心收益；对实时交互任务，latency 仍需作为独立约束	讨论与结论章节

路由模式级结论

Throughput First

指标	Infron / OpenRouter 并列对比	胜出方
缓存命中率	Infron 97.41% OpenRouter 93.54%	Infron (高 4.14%)
实际成本	Infron \$0.36936000 OpenRouter \$3.59489664	Infron (低 89.73%)
Throughput	Infron 2.67 tok/s OpenRouter 3.39 tok/s	OpenRouter (高 26.62%)
Latency	Infron 5980.69 ms OpenRouter 4723.73 ms	OpenRouter (低 21.02%)
TTFT	Infron 5786.47 ms OpenRouter 4423.83 ms	OpenRouter (低 23.55%)

Price First

指标	Infron / OpenRouter 并列对比	胜出方
缓存命中率	Infron 98.58% OpenRouter 66.56%	Infron (高 48.10%)
实际成本	Infron \$0.35053300 OpenRouter \$0.72619654	Infron (低 51.73%)
Throughput	Infron 2.60 tok/s OpenRouter 1.79 tok/s	Infron (高 45.15%)
Latency	Infron 6150.60 ms OpenRouter 8930.26 ms	Infron (低 31.13%)
TTFT	Infron 5959.01 ms OpenRouter 8499.25 ms	Infron (低 29.89%)

Latency First

指标	Infron / OpenRouter 并列对比	胜出方
缓存命中率	Infron 98.43% OpenRouter 98.90%	OpenRouter (高 0.48%)
实际成本	Infron \$0.39538500 OpenRouter \$3.53757760	Infron (低 88.82%)
Throughput	Infron 5.30 tok/s OpenRouter 5.49 tok/s	OpenRouter (高 3.47%)
Latency	Infron 3016.90 ms OpenRouter 2916.05 ms	OpenRouter (低 3.34%)
TTFT	Infron 2837.35 ms OpenRouter 2564.25 ms	OpenRouter (低 9.63%)

TTFT First

指标	Infron / OpenRouter 并列对比	胜出方
缓存命中率	Infron 98.43% OpenRouter 98.88%	OpenRouter (高 0.46%)
实际成本	Infron \$0.37421000 OpenRouter \$3.63688800	Infron (低 89.71%)
Throughput	Infron 4.88 tok/s OpenRouter 5.56 tok/s	OpenRouter (高 14.13%)
Latency	Infron 3281.74 ms OpenRouter 2875.68 ms	OpenRouter (低 12.37%)
TTFT	Infron 2994.02 ms OpenRouter 2440.43 ms	OpenRouter (低 18.49%)

说明：每个区块对应一种路由模式；同一指标行内的 Infron 与 OpenRouter 柱条按两者最大值归一化。缓存命中率和 throughput 越高越好，实际成本、latency 和 TTFT 越低越好。

1. 引言：背景、研究问题与贡献

本实验评估同一 OpenAI-compatible Chat Completions 请求在 Infron 与 OpenRouter 两个平台上的 prompt caching 表现。评估重点是：在输入条件严格一致时，不同 provider routing sort 策略会如何影响缓存命中、实际成本、吞吐量和端到端时延。

Prompt caching 对生产业务的核心价值在于：当业务请求包含稳定系统提示词、长上下文模板、RAG 前缀、工具说明或固定工作流指令时，第二次及后续请求理论上可以复用已处理的输入 token，从而降低单位请求成本，并可能改善整体服务稳定性。本实验通过“两次相同 prompt 请求”的方式构造可重复观测场景，用第二次请求的 cache read tokens 衡量缓存收益。

本报告回答三个问题：第一，在相同 payload 和 `usage.prompt_tokens` 偏差不超过 50 tokens 的口径下，Infron 与 OpenRouter 的缓存命中和成本表现有何差异；第二，不同 routing sort（`throughput`、`price`、`latency`、`ttft`）下速度、成本、首包和缓存如何变化；第三，从可观测 telemetry 看，两个平台的路由选择如何影响最终结果。由于 OpenRouter 不支持 `provider.sort=ttft`，TTFT First 的 A/B 设计为 Infron `sort=ttft` 对比 OpenRouter `sort=latency`。

1.1 研究假设

假设	内容	验证指标
H1	在重复稳定长前缀请求中，更强的 provider/cache affinity 会提升 Token 级缓存命中率	第二次请求 cache read tokens、Token 级命中率
H2	更高缓存命中率会降低真实响应成本，但不必然降低 TTFT 或端到端 latency	实际成本、平均 TTFT、平均 latency/请求
H3	不同 routing sort 会改变 provider 选择，从而形成不同的成本、吞吐和时延 Pareto 前沿	provider 分布、throughput、latency、cost

1.2 本文贡献

- 给出一个严格配对的 A/B benchmark 方法，使用响应返回的 `usage.prompt_tokens` 作为真实 input token 控制变量。
- 将 prompt caching 评估从单一 cache hit 指标扩展到成本、吞吐、latency、TTFT、provider 分布和可复现数据集。
- 支持按 prompt 长度 tier 分层观察 cache read tokens、Token 级命中率和成本，让缓存收益不再只看总体均值。
- 用可观测 telemetry 解释 Infron 与 OpenRouter 的路由差异，同时明确内部 routing trace 缺失时的归因边界。
- 提供配对级 CSV、请求级 JSONL 和 A/B testing 代码，便于后续重复实验和第三方审计。

2. 方法：实验设计、数据集构造与控制变量

2.1 数据集生成方法

实验数据集由脚本自动生成，共覆盖 4 种 routing sort、2 个平台、4 个实验组、每组 50 轮。每一轮包含两次完全相同的 chat/completions 请求：第一次用于建立或触发缓存写入，第二次用于观测缓存读取。每个逻辑 routing sort 都记录平台侧实际 payload 的 SHA256，以便验证请求内容没有漂移。

Prompt 构造采用稳定长前缀加固定用户输入：system message 包含重复的 cache probe prefix，user message 固定为 Reply with exactly: cache probe ok。模型、temperature、max_tokens、usage include、provider sort 等参数在同一 routing sort 内保持不变。该数据集用于测量 prompt caching 行为，不代表业务语料分布。本轮同时启用 prompt 长度分层：short ≈ 1500 tokens，medium ≈ 8000 tokens，long ≈ 32000 tokens；脚本按 group/round 稳定分配 tier。

2.2 控制变量方法

A/B 测试的基本配对单元是同一 sort/group/round 下的 Infron 记录和 OpenRouter 记录。只有当两边 first request 与 second request 的 usage.prompt_tokens 各自偏差不超过 50 tokens 时，该配对才进入最终统计；任何 HTTP 非 200、请求异常、usage.prompt_tokens ≤ 0 或 A/B 输入 token 偏差超过阈值的记录都会被剔除。这保证了成本、缓存命中率、吞吐量和时延的对比建立在相近输入规模上。

本报告中的总 Input Tokens 严格取自响应返回的 usage.prompt_tokens，不使用本地 tokenizer 估算值。原因是 provider 的真实处理、缓存和计费口径最终以响应 usage 为准。通过使用响应 usage 并执行 A/B 配对一致性过滤，实验避免了 tokenizer 差异、服务端 prompt 包装和异常 usage 上报带来的偏差。

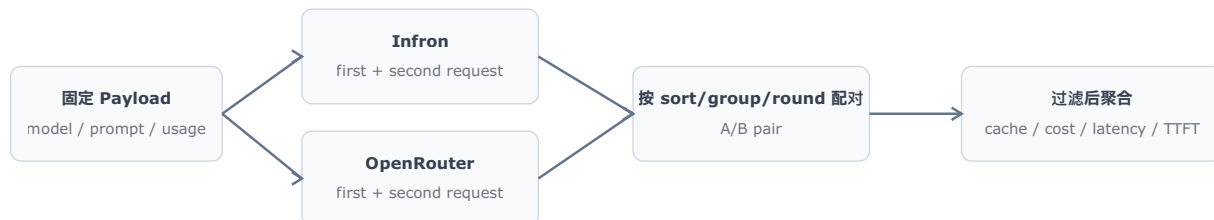
本轮启用 prompt 长度分层测试，分层计划为 short ≈ 1500 tokens、medium ≈ 8000 tokens、long ≈ 32000 tokens。分层只改变稳定 prefix 的目标长度，不改变模型、temperature、max_tokens、routing sort、first/second 双请求、A/B input token 容差和 telemetry 口径；同一 sort/group/round 下 Infron 与 OpenRouter 仍发送同一 tier 的同源 messages。

2.3 实验设置图示与代码示例

下图展示单个 routing sort 下的实验流水线：同一 payload 分别发送到 Infron 与 OpenRouter，每个平台每轮连续发送两次相同请求，最终在同一 sort/group/round 维度做严格 A/B 配对。

实验数据生成流程

同一 payload、同一 routing sort、同一 group/round 下分别请求 Infron 与 OpenRouter。



每轮请求两次相同 prompt：第一次触发/建立缓存，第二次观测 cache read tokens。

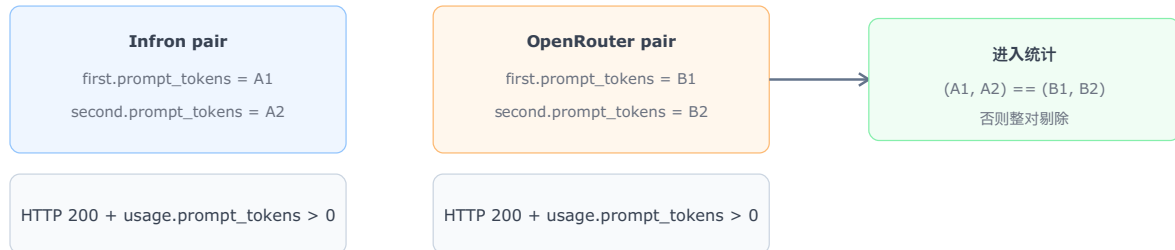
同一 routing sort 下 payload SHA256 固定，用响应 usage.prompt_tokens 做真实 token 口径。

图 1: 实验流水线。该图强调每个 routing sort 下的同源 payload、双平台请求和 first/second request 配对关系，用于说明实验如何构造可比样本。

A/B 配对过滤的目标是确保比较只发生在输入 token 规模足够接近的样本上。只有 first request 与 second request 的 `usage.prompt_tokens` 在两边各自偏差不超过 50 tokens，样本才进入最终统计。

A/B 配对与控制变量过滤

只保留 first/second prompt tokens 在 A/B 两边偏差不超过 50 的配对样本。



该过滤防止 tokenization、服务端包装、异常 usage 上报对成本、缓存命中和性能指标造成混杂偏差。

图 2: A/B 配对过滤逻辑。该图明确展示异常 usage、HTTP 异常、非完整配对和 input tokens 不一致样本如何被排除，保证最终对比符合控制变量要求。

核心请求 payload 结构如下。实验固定模型、温度、最大输出 token、usage 返回和 provider sort，只改变路由优先模式。

```
{
  "model": "minimax/minimax-m2.7",
  "messages": [
    {"role": "system", "content": "<stable long cache probe prefix>"},
    {"role": "user", "content": "Reply with exactly: cache probe ok"}
  ],
  "temperature": 0,
  "max_tokens": 16,
  "usage": {"include": true},
  "provider": {"sort": "throughput | price | latency | ttft", "allow_fallbacks": true}
}
```

TTFT First 对照规则：Infron 请求使用 `provider.sort=ttft`；OpenRouter 不支持该参数，因此 OpenRouter 请求使用 `provider.sort=latency` 作为首包/时延优先对照。报告中的逻辑路由模式仍统一记为 `ttft`，用于配对、聚合和可视化。

最终过滤逻辑可概括为以下伪代码。这个步骤是本实验控制变量的核心。

```
for pair in group_by(records, key=(sort, group, round)):
    infron = pair['infron']
    openrouter = pair['openrouter']
    if not both_http_200(infron, openrouter):
        exclude(pair)
    elif any(request.usage.prompt_tokens <= 0 for request in pair.requests):
        exclude(pair)
    elif (infron.first.prompt_tokens, infron.second.prompt_tokens) !=
         (openrouter.first.prompt_tokens, openrouter.second.prompt_tokens):
        exclude(pair)
```



```
else:
    include(pair)
```

2.4 指标定义

表 1：核心指标定义与解释方向。

指标	定义	解释方向
调用级命中率	第二次请求 <code>cache_read_tokens > 0</code> 的轮次占比	越高表示越稳定触发缓存读取
Token 级命中率	第二次请求 <code>cache read tokens / 第二次请求 prompt tokens</code>	越高表示输入 token 复用比例越高
实际成本	<code>first + second</code> 两次请求返回 <code>usage/cost</code> 的合计	越低越好，代表真实账单风险更低
平均 throughput	响应 <code>completion tokens / 请求 latency seconds</code> ； <code>reasoning tokens</code> 作为响应 <code>usage</code> 组成部分处理，不单独拆成独立 KPI	越高越好，代表单位时间响应输出能力更强
平均 latency/ 请求	每次请求完整响应耗时均值	越低越好，代表用户等待时间更短
平均 TTFT	streaming 下首包/首 token 到达时间均值	越低越好，代表用户更快看到首个响应信号
Reasoning 口径	响应 <code>usage</code> 中的 <code>reasoning token</code> 字段作为响应统计的组成部分保留在原始记录和 <code>summary</code> 中	不单独展示排名，避免把内部推理预算误读为独立业务产出
TTFT	首 token 到达时间	本轮已启用 streaming 并采集 TTFT；TTFT 与完整响应 <code>latency</code> 分别代表首 token 体验和完整响应体验

2.5 表格、图表与架构图表达规范

为了让报告更容易审计，表格、图表和架构图采用统一表达方式：表格负责精确数值比较，趋势图展示指标变化过程，架构图解释机制假设与可观测证据之间的关系。结论以响应 `telemetry` 为准，架构图只用于解释机制。

表 2：可视化与表格专业性评估。

类型	当前用途	专业性评估	后续可补充项
总览表	展示核心指标、胜出方和可比样本规模	保留精确数值、单位和胜出高亮，适合审计；本轮报告已加入 <code>bootstrap CI</code> 与 <code>paired permutation test</code>	已补充： <code>bootstrap CI</code> 、 <code>p-value</code> ；后续可加入 <code>standardized effect size</code>
分组明细表	检查不同 group 的稳定性	能发现单组异常和策略漂移；本轮报告已加入 <code>P50/P95/P99 latency/TTFT</code>	已补充： <code>P50/P95/P99</code> ；后续可加入 <code>IQR</code> 和 <code>tail amplification</code>
核心指标柱状图	按 <code>routing mode</code> 对比 <code>latency</code> 、 <code>throughput</code> 、 <code>cost</code> 、 <code>cache hit rate</code>	适合快速判断胜出方和指标差异；后续可增加误差棒	待补充： <code>error bar</code> 、 <code>confidence band</code> 可视化
指标生成曲线	展示每组请求的指标变化过程	有助于观察缓存预热、波动和异常点；后续可加入事件标注	待补充： <code>warm-up annotation</code> 、 <code>outlier labels</code>

类型	当前用途	专业性评估	后续可补充项
架构图	解释 Infron provider routing、provider stick 和成本控制机制	明确区分可观测证据与机制解释，避免把内部实现假设误写成事实	待补充：真实 routing trace、provider cost breakdown 明细

3. 实验环境与数据质量控制

表 3：实验配置与数据质量控制规则。

项目	配置
测试模型	minimax/minimax-m2.7
对比平台	Infron、OpenRouter
路由偏好	throughput、price、latency、ttft
TTFT First 对照	Infron 使用 provider.sort=ttft；OpenRouter 使用 provider.sort=latency
数据集名称	controlled_cache_probe
数据集类型	Controlled synthetic prompt-cache probe with stable long prefix
外部业务语料	未提供；本轮使用脚本内置/合成数据集
实验组数	每个平台每种路由 4 组
每组轮次	50 轮
并发 worker 数	8
长稳运行目标	0 秒
本地代理控制	同一本地代理：是；代理：启用 socks5://127.0.0.1:1086；隐式环境代理：已禁用
每轮请求	两次相同 prompt 请求，用第二次请求统计缓存命中
Usage 采集	请求默认带 usage: {"include": true}，以响应 usage 作为真实统计口径
成本口径	只统计响应真实返回的 usage.cost 或 cost breakdown；若平台未返回成本字段，则显示 N/A，不按 0 计入胜负
Reasoning / Thinking 控制	请求未显式指定 reasoning effort，保留模型与平台默认 thinking/reasoning 行为；响应 usage 中的 reasoning tokens 作为观测
Streaming / TTFT 采集	已启用 streaming，并记录 TTFT/首内容 token/首 reasoning token 时间
Provider 归因采集	脚本记录响应 headers、response model/id/system fingerprint、provider/routing trace 候选字段、provider cost breakdown
结果目录	export/minimax_m27_all_experiments/ routing_sort_cache_cost_ab_4x50_stream_ttft_reasoning_default_length_tiers_chat_complet.
剔除规则	HTTP 非 200、请求异常、任一请求 usage.prompt_tokens <= 0、或同一 sort/group/round 下 A/B 两边 first/usage.prompt_tokens 偏差超过 50 tokens 的轮次不进入统计
剔除记录数	0 条
throughput payload SHA256	Infron 3e7abdcf47c6d12d2e1cf8ed3015baadd0ccf075d484369a619df67273565c79 OpenRouter 3e7abdcf47c6d12d2e1cf8ed3015baadd0ccf075d484369a619df67273565c79

项目	配置
price payload SHA256	Infron c3a31e96d59b532e90d05c9b714dfe78b13e539f1c01384487c9041d12df7326 OpenRouter c3a31e96d59b532e90d05c9b714dfe78b13e539f1c01384487c9041d12df7326
latency payload SHA256	Infron db87a04a52980e8d61212ec01620c987dcab9d73acd9a18f8caafe304495cf62 OpenRouter db87a04a52980e8d61212ec01620c987dcab9d73acd9a18f8caafe304495cf62
ttft payload SHA256	Infron b7fe21359fe7c46654cfc1ee3edd38ba464fa2c0c9fc9a1a0fa3d08d6ee2d1a OpenRouter db87a04a52980e8d61212ec01620c987dcab9d73acd9a18f8caafe304495cf62

说明：A/B 控制变量是同一 routing sort 下发送给 Infron 和 OpenRouter 的请求 payload。总览中的 Input Tokens 按响应返回的 usage.prompt_tokens 汇总，代表各平台实际统计和计费口径下处理的输入 token 量。

4. 结果：总体指标与主要发现

说明：本节的 throughput、latency 和 TTFT 均为响应级整体指标。若响应 usage 中 completion_tokens 包含 reasoning tokens，则 reasoning 过程已纳入 throughput 分子；请求 latency 是完整响应端到端耗时，天然包含 reasoning 过程耗时；TTFT 是 streaming 下首个 SSE token/chunk 到达时间，代表首包响应体验。成本只使用响应明确返回的 cost 字段；未返回 cost 时标记为 N/A，不视为 0。

表 4：总体 A/B 指标对比。加粗单元表示同一 routing sort 下表现更好的一方；Input Tokens 加粗表示两边严格相等。

路由偏好	平台	总 轮 数	成 功 轮 数	总 Input Tokens (usage.prompt_tokens)	调用级命 中率	Token 级命中 率	实际总成本	平均每轮成本	平均响应 throughput (含 reasoning)
throughput	Infron	200	200	6207102	99.50%	97.41%	\$0.36936000	\$0.00184680	2.67 response tok/s
throughput	OpenRouter	200	200	6207808	97.00%	93.54%	\$3.59489664	\$0.01797448	3.39 response tok/s
price	Infron	200	200	6207102	100.00%	98.58%	\$0.35053300	\$0.00175266	2.60 response tok/s
price	OpenRouter	200	200	6207808	65.50%	66.56%	\$0.72619654	\$0.00363098	1.79 response tok/s
latency	Infron	200	200	6207008	100.00%	98.43%	\$0.39538500	\$0.00197693	5.30 response tok/s

路由偏好	平台	总 轮 数	成 功 轮 数	总 Input Tokens (<code>usage.prompt_tokens</code>)	调用级命 中率	Token 级命中 率	实际总成本	平均每轮成本	平均响应 throughput (含 reasoning)
latency	OpenRouter	200	200	6207808	100.00%	98.90%	\$3.53757760	\$0.01768789	5.49 response tok/s
ttft	Infron	200	200	6207008	100.00%	98.43%	\$0.37421000	\$0.00187105	4.88 response tok/s
ttft	OpenRouter	200	200	6207808	100.00%	98.88%	\$3.63688800	\$0.01818444	5.56 response tok/s

4.1 尾延迟与显著性检验

表 5：尾延迟分位数。P95/P99 直接从请求级 latency 与 TTFT 计算，补充均值无法表达的尾部风险。

路由偏好	平台	P50 Latency	P95 Latency	P99 Latency	P50 TTFT	P95 TTFT	P99 TTFT
throughput	Infron	5510.77 ms	10431.51 ms	13793.99 ms	5312.49 ms	10017.12 ms	13588.22 ms
throughput	OpenRouter	4362.23 ms	9145.17 ms	12017.44 ms	4017.70 ms	8722.63 ms	10439.15 ms
price	Infron	5373.54 ms	10261.29 ms	18709.11 ms	5045.71 ms	10073.32 ms	18519.32 ms
price	OpenRouter	7150.14 ms	21515.00 ms	31077.30 ms	6707.25 ms	21236.52 ms	30497.70 ms
latency	Infron	2676.52 ms	4857.49 ms	7348.92 ms	2485.94 ms	4315.27 ms	7194.22 ms
latency	OpenRouter	2410.66 ms	4450.04 ms	17604.58 ms	2040.22 ms	3793.96 ms	16747.76 ms
ttft	Infron	2986.60 ms	5337.22 ms	7932.95 ms	2711.70 ms	4742.16 ms	7773.80 ms
ttft	OpenRouter	2582.83 ms	5296.03 ms	7541.27 ms	2171.05 ms	4429.90 ms	6461.64 ms

表 6：配对统计检验。均值差使用 bootstrap 95% CI, p-value 使用 paired sign-flip permutation test。指标名给出差值方向，解释列说明正值代表的含义。

路由偏好	指标	均值差	95% CI	p-value	配对 数	解释
throughput	Latency: OpenRouter – Infron	–2513.92 ms	[–3168.92 ms, –1829.95 ms]	<0.001	200	正值表示 Infron latency 更低
throughput	TTFT: OpenRouter – Infron	–2725.28 ms	[–3374.37 ms, –2041.87 ms]	<0.001	200	正值表示 Infron TTFT 更低
throughput	Throughput: Infron – OpenRouter	–1.1193 tok/s	[–1.4014 tok/s, –0.8388 tok/s]	<0.001	200	正值表示 Infron throughput 更高
throughput	Cost: OpenRouter – Infron	\$0.01612768	[\$0.01387616, \$0.01837601]	<0.001	200	正值表示 Infron 成本更低
throughput	Token Cache Hit: Infron – OpenRouter	–0.25 pp	[–2.76 pp, 2.60 pp]	0.8658	200	正值表示 Infron cache hit 更高

路由偏好	指标	均值差	95% CI	p-value	配对数	解释
price	Latency: OpenRouter – Infron	5559.31 ms	[3784.12 ms, 7392.94 ms]	<0.001	200	正值表示 Infron latency 更低
price	TTFT: OpenRouter – Infron	5080.49 ms	[3310.00 ms, 6915.62 ms]	<0.001	200	正值表示 Infron TTFT 更低
price	Throughput: Infron – OpenRouter	0.7524 tok/s	[0.5433 tok/s, 0.9857 tok/s]	<0.001	200	正值表示 Infron throughput 更高
price	Cost: OpenRouter – Infron	\$0.00187832	[\$0.00150270, \$0.00227763]	<0.001	200	正值表示 Infron 成本更低
price	Token Cache Hit: Infron – OpenRouter	29.83 pp	[22.99 pp, 36.72 pp]	<0.001	200	正值表示 Infron cache hit 更高
latency	Latency: OpenRouter – Infron	−201.71 ms	[−958.51 ms, 706.30 ms]	0.6536	200	正值表示 Infron latency 更低
latency	TTFT: OpenRouter – Infron	−546.21 ms	[−1303.57 ms, 381.92 ms]	0.2107	200	正值表示 Infron TTFT 更低
latency	Throughput: Infron – OpenRouter	−0.8461 tok/s	[−1.0991 tok/s, −0.6004 tok/s]	<0.001	200	正值表示 Infron throughput 更高
latency	Cost: OpenRouter – Infron	\$0.01571096	[\$0.01342238, \$0.01801199]	<0.001	200	正值表示 Infron 成本更低
latency	Token Cache Hit: Infron – OpenRouter	−3.78 pp	[−4.50 pp, −3.05 pp]	<0.001	200	正值表示 Infron cache hit 更高
ttft	Latency: OpenRouter – Infron	−812.11 ms	[−1330.15 ms, −306.48 ms]	<0.001	200	正值表示 Infron latency 更低
ttft	TTFT: OpenRouter – Infron	−1107.17 ms	[−1569.67 ms, −695.49 ms]	<0.001	200	正值表示 Infron TTFT 更低
ttft	Throughput: Infron – OpenRouter	−0.8220 tok/s	[−1.0758 tok/s, −0.5458 tok/s]	<0.001	200	正值表示 Infron throughput 更高
ttft	Cost: OpenRouter – Infron	\$0.01631339	[\$0.01411223, \$0.01858295]	<0.001	200	正值表示 Infron 成本更低
ttft	Token Cache Hit: Infron – OpenRouter	−3.76 pp	[−4.53 pp, −3.04 pp]	<0.001	200	正值表示 Infron cache hit 更高

4.2 Reasoning / Thinking 控制校验

表 7: Reasoning telemetry 观测。本轮未显式指定 reasoning/thinking 参数，保留模型与平台默认行为；该表用于记录默认行为下的 reasoning tokens 和首 reasoning token 观测。

路由偏好	平台	Reasoning Tokens	平均 Reasoning Tokens/请求	Reasoning 请求数	平均首 Reasoning Token	平均 TTFT	平均端到端 E2E 时延
throughput	Infron	5643	14.1075	353	5799.21 ms	5786.47 ms	5980.69 ms

路由偏好	平台	Reasoning Tokens	平均 Reasoning Tokens/请求	Reasoning 请求数	平均首 Reasoning Token	平均 TTFT	平均端到端 E2E 时延
throughput	OpenRouter	7879	19.6975	400	4423.83 ms	4423.83 ms	4723.73 ms
price	Infron	5635	14.0875	353	5970.70 ms	5959.01 ms	6150.60 ms
price	OpenRouter	7015	17.5375	400	8499.25 ms	8499.25 ms	8930.26 ms
latency	Infron	6389	15.9725	400	2837.35 ms	2837.35 ms	3016.90 ms
latency	OpenRouter	7981	19.9525	400	2564.25 ms	2564.25 ms	2916.05 ms
ttft	Infron	6389	15.9725	400	2994.02 ms	2994.02 ms	3281.74 ms
ttft	OpenRouter	7960	19.9000	400	2440.43 ms	2440.43 ms	2875.68 ms

本轮 Reasoning / Thinking 控制：请求未显式指定 reasoning effort，保留模型与平台默认 thinking/reasoning 行为；响应 usage 中的 reasoning tokens 作为观测变量记录。

4.3 API 协议记录

本轮 API 协议为 /v1/chat/completions；本表记录两家平台在该协议下的 HTTP 成功、usage、token usage、成本和缓存 telemetry 覆盖。加粗单元表示覆盖率更高的一方。

API 协议	Endpoint	平台	配对轮数	请求数	成功率	Usage 覆盖	Token Usage 覆盖	成本覆盖	缓存 Telemetry 覆盖	HTTP 状态
chat_completions	/v1/chat/completions	Infron	800	1600	100.00%	100.00%	100.00%	100.00%	100.00%	{"200", 1600}
chat_completions	/v1/chat/completions	OpenRouter	800	1600	100.00%	100.00%	100.00%	100.00%	100.00%	{"200", 1600}

5. 结果可视化：按路由模式的核心指标变化

说明：本节按路由模式组织图表。每张图对应一种 First 路由模式，并在同一图内对比 Infron 与 OpenRouter 的 latency、TTFT、throughput、实际成本和 Token 级缓存命中率，方便观察同一模式下的 A/B 指标差异。本轮已启用 streaming，并采集 TTFT、首内容 token 与首 reasoning token 时间；TTFT 代表首包响应体验，latency 代表完整响应体验。

Throughput First 路由模式

Throughput First 路由模式：核心指标 A/B 对比

每个面板均比较 Infron 与 OpenRouter；胜出方使用浅色底、粗描边和右上角标签突出展示。

Infron
OpenRouter

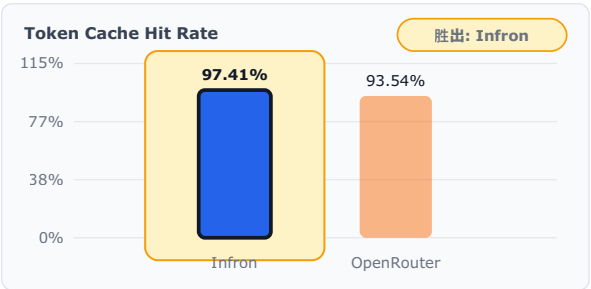
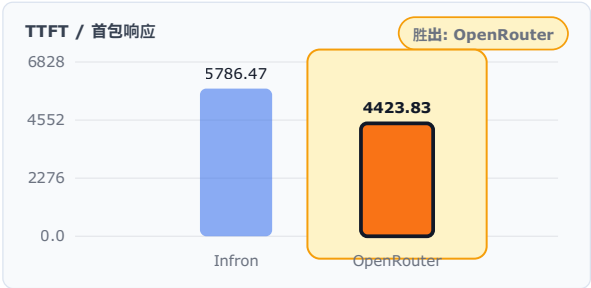
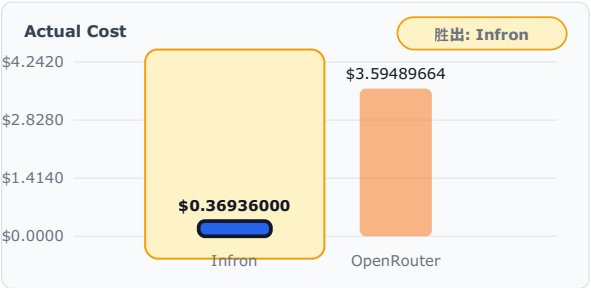
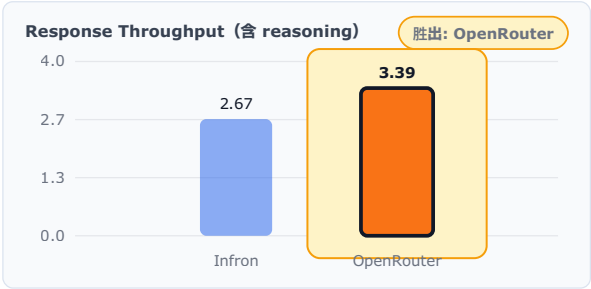
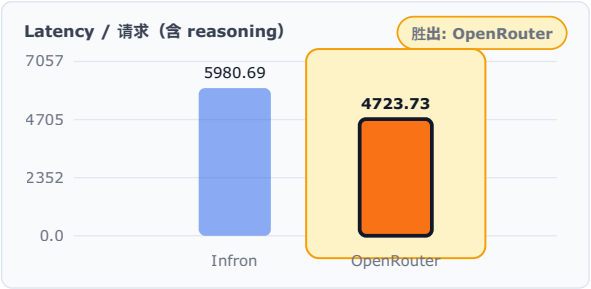


图 3：Throughput First 路由模式下的核心指标对比。柱状图同时呈现缓存、成本、吞吐、TTFT 和 latency 表现。

Throughput First 路由模式：综合雷达图

所有轴都归一化为“越外圈越好”：成本、时延、TTFT 已做反向评分。

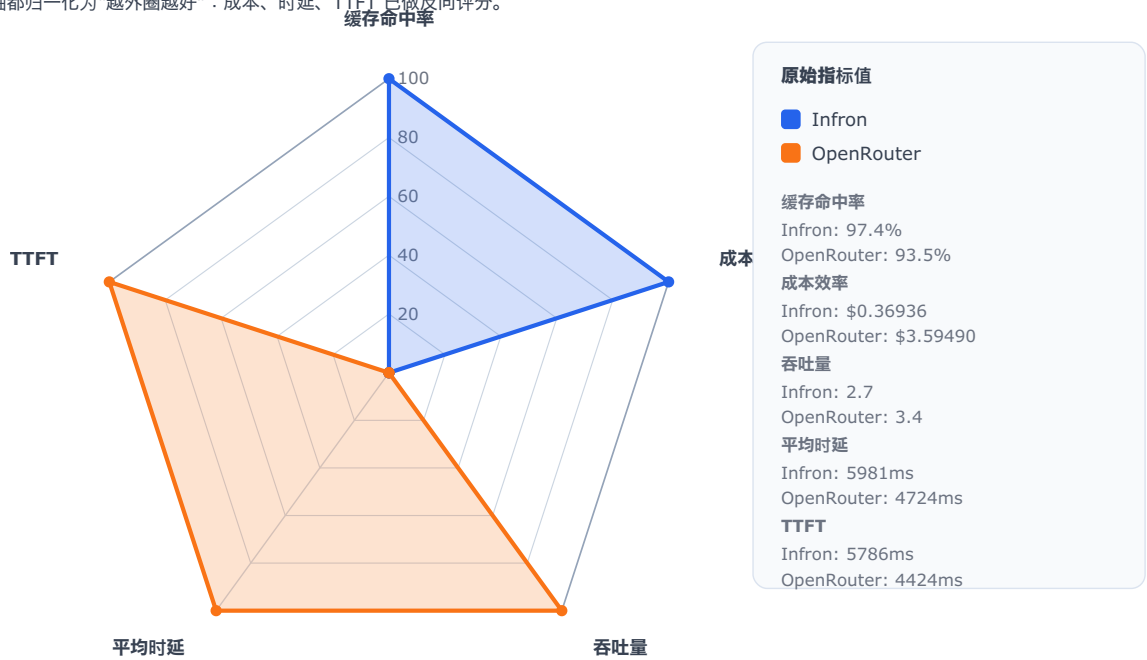


图 4：Throughput First 路由模式下的综合雷达图。所有轴都按“越外圈越好”归一化，便于快速比较两家平台的综合形状。

Throughput First 路由模式：指标生成过程对比曲线

按 group/round 顺序绘制每轮指标；曲线展示汇总均值背后的波动过程。

Infron
OpenRouter

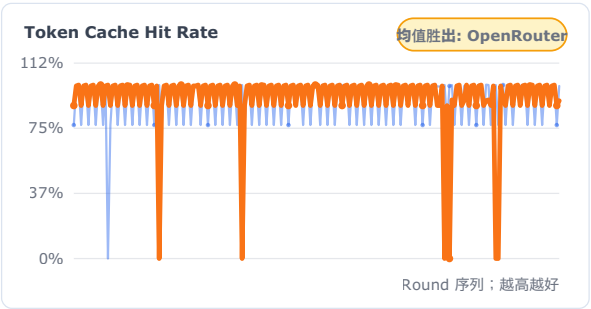
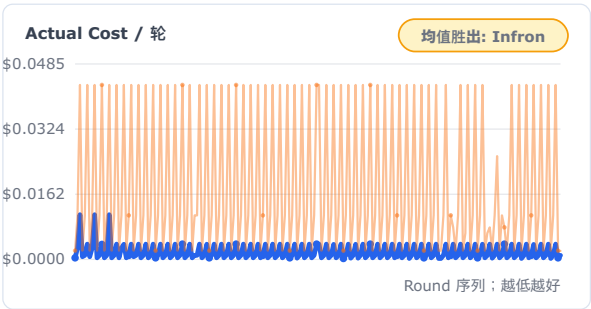
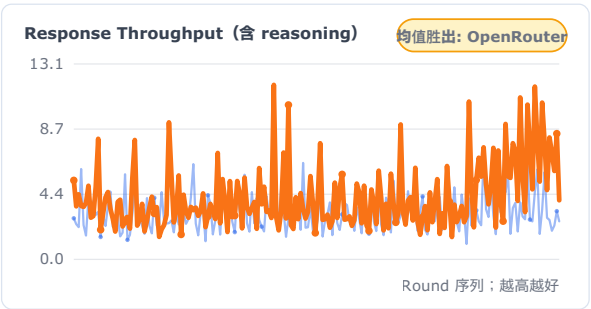
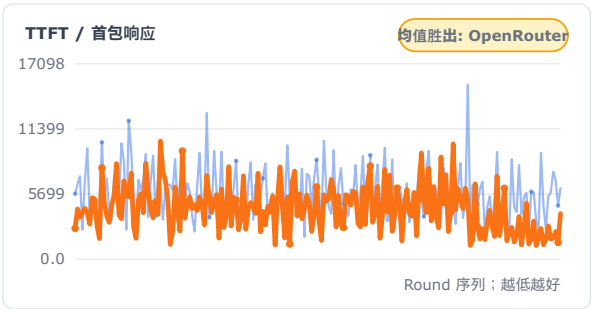
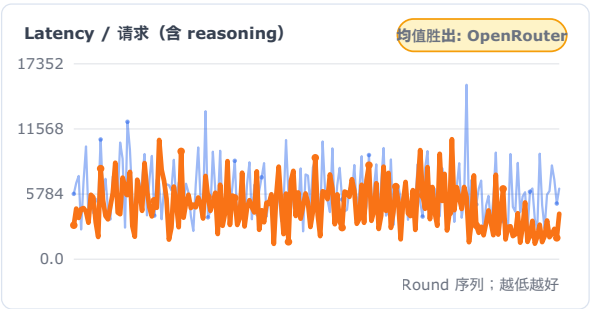


图 5：Throughput First 路由模式下的指标生成过程曲线。该图用于观察指标随 group/round 的变化，而不只依赖均值。

Price First 路由模式

Price First 路由模式：核心指标 A/B 对比

每个面板均比较 Infron 与 OpenRouter；胜出方使用浅色底、粗描边和右上角标签突出展示。

Infron
OpenRouter

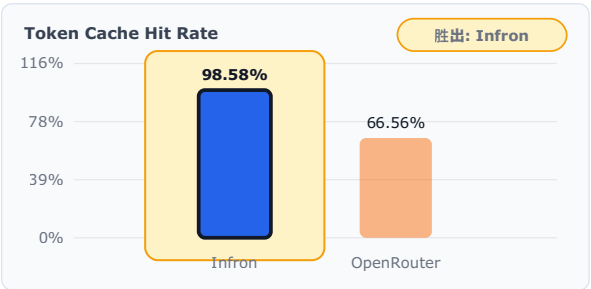
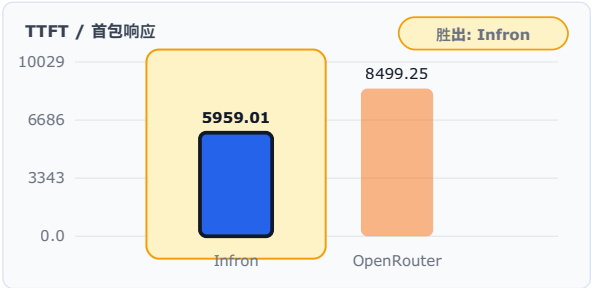
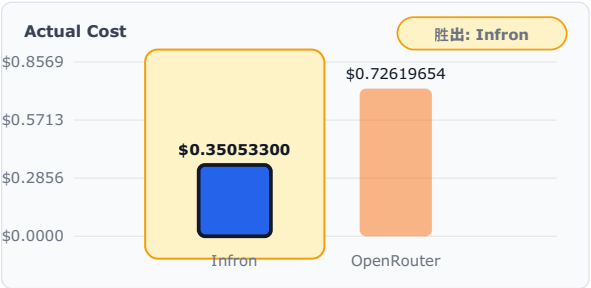
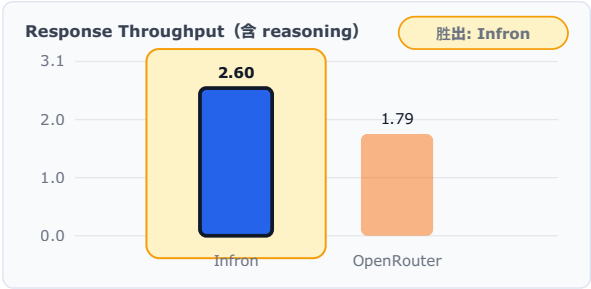
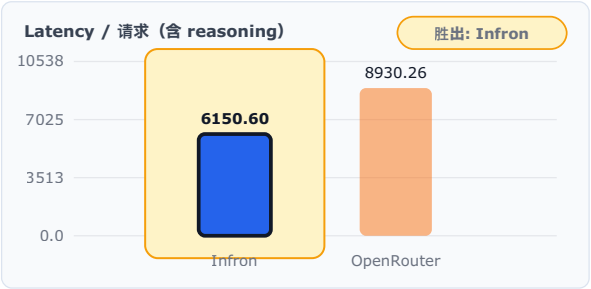


图 6：Price First 路由模式下的核心指标对比。柱状图同时呈现缓存、成本、吞吐、TTFT 和 latency 表现。

Price First 路由模式：综合雷达图

所有轴都归一化为“越外圈越好”：成本、时延、TTFT 已做反向评分。



图 7：Price First 路由模式下的综合雷达图。所有轴都按“越外圈越好”归一化，便于快速比较两家平台的综合形状。

Price First 路由模式：指标生成过程对比曲线

按 group/round 顺序绘制每轮指标；曲线展示汇总均值背后的波动过程。



图 8：Price First 路由模式下的指标生成过程曲线。该图用于观察指标随 group/round 的变化，而不只依赖均值。

Latency First 路由模式

Latency First 路由模式：核心指标 A/B 对比

每个面板均比较 Infron 与 OpenRouter；胜出方使用浅色底、粗描边和右上角标签突出展示。

Infron
OpenRouter

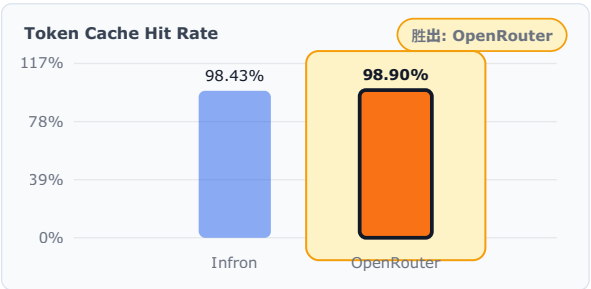
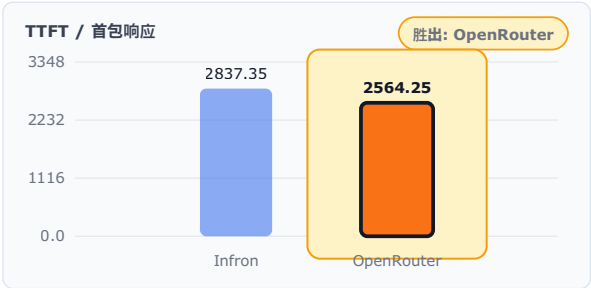
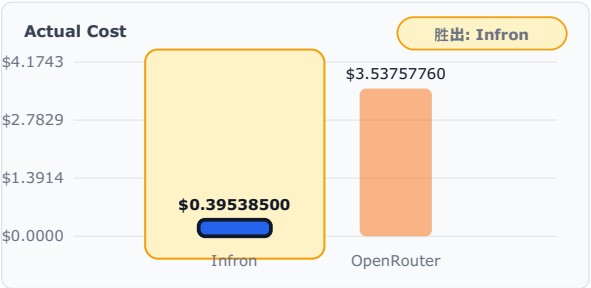
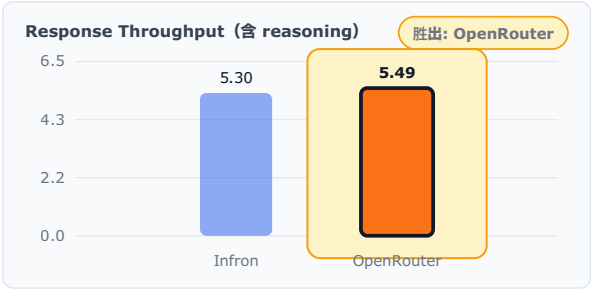
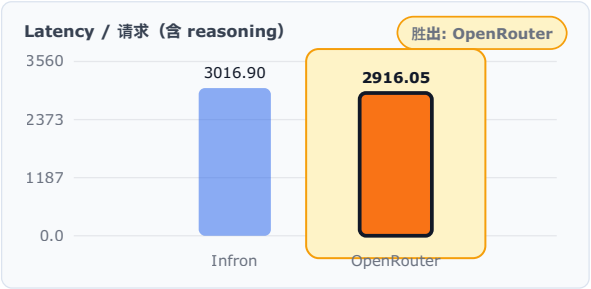


图 9：Latency First 路由模式下的核心指标对比。柱状图同时呈现缓存、成本、吞吐、TTFT 和 latency 表现。

Latency First 路由模式：综合雷达图

所有轴都归一化为“越外圈越好”：成本、时延、TTFT 已做反向评分。

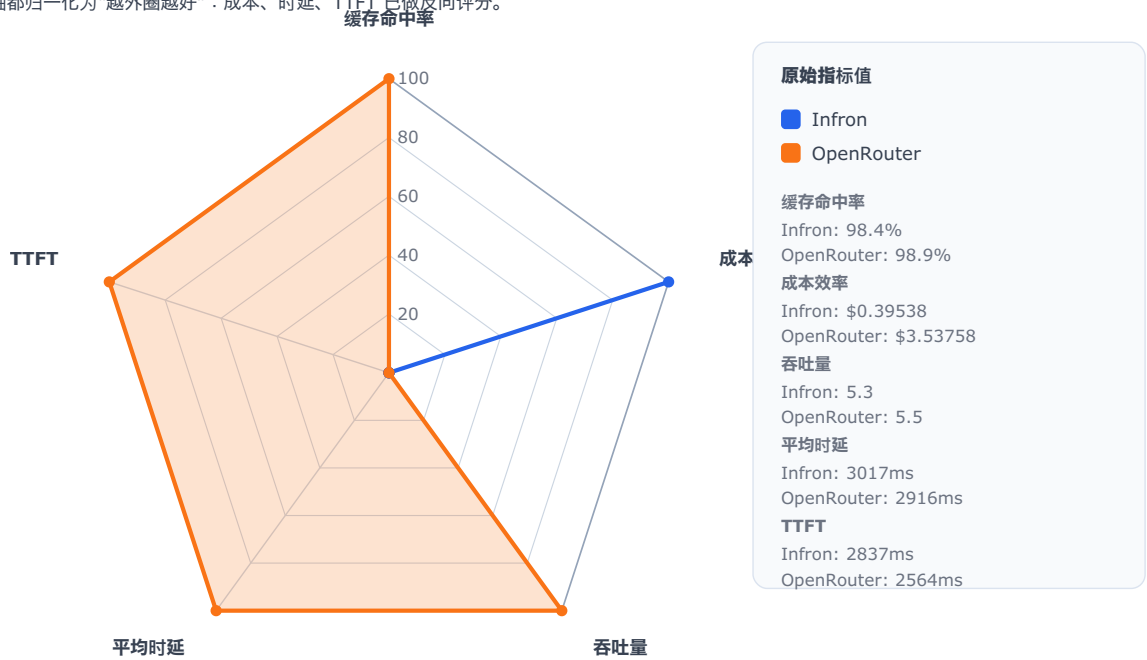


图 10：Latency First 路由模式下的综合雷达图。所有轴都按“越外圈越好”归一化，便于快速比较两家平台的综合形状。

Latency First 路由模式：指标生成过程对比曲线

按 group/round 顺序绘制每轮指标；曲线展示汇总均值背后的波动过程。

Infron
OpenRouter

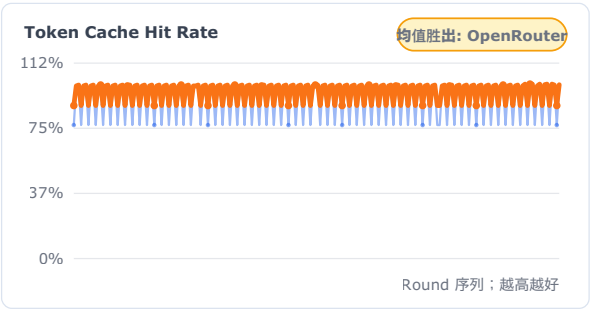
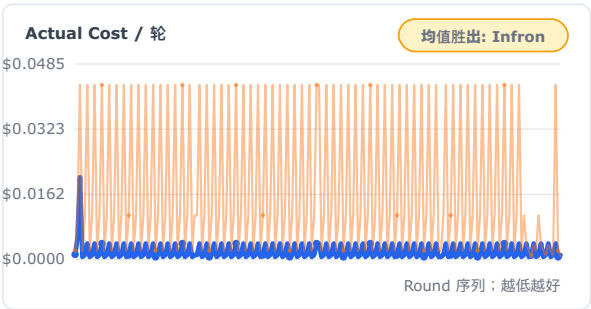
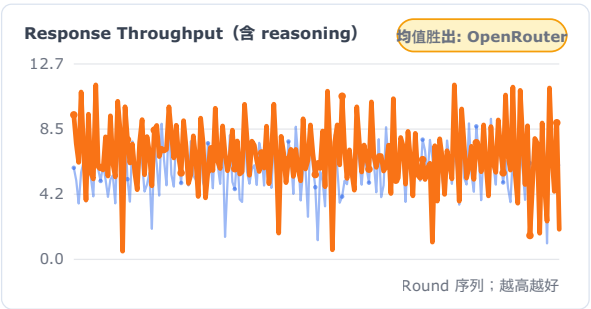
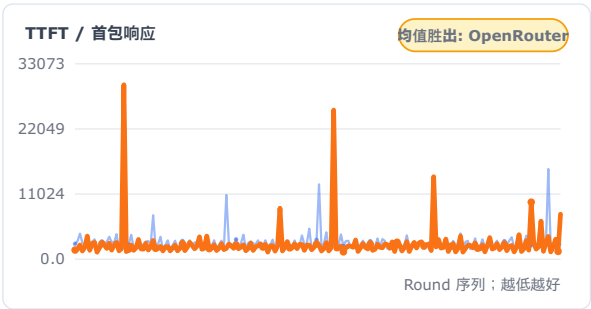
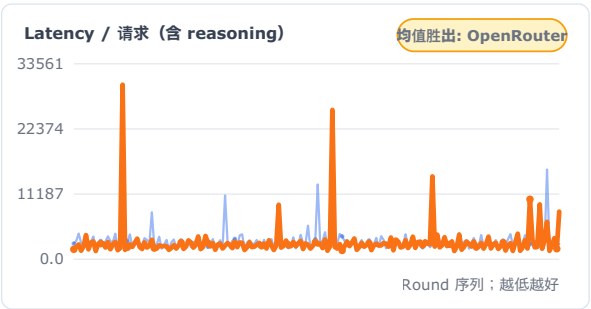


图 11: Latency First 路由模式下的指标生成过程曲线。该图用于观察指标随 group/round 的变化，而不只依赖均值。

TTFT First 路由模式

TTFT First 路由模式：核心指标 A/B 对比

每个面板均比较 Infron 与 OpenRouter；胜出方使用浅色底、粗描边和右上角标签突出展示。

Infron
OpenRouter

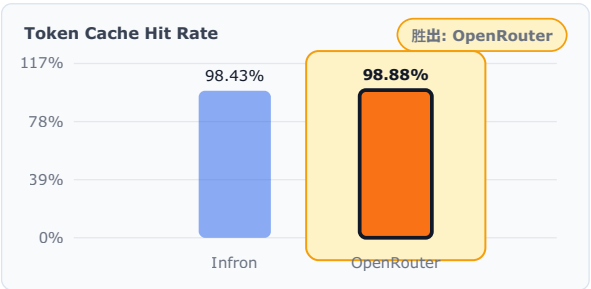
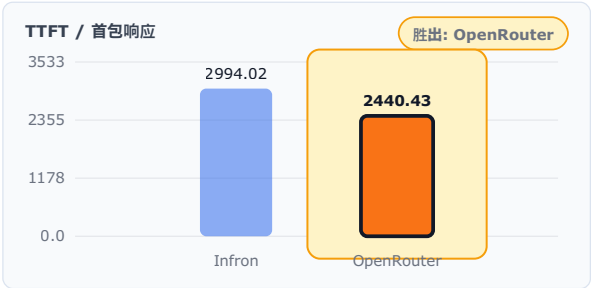
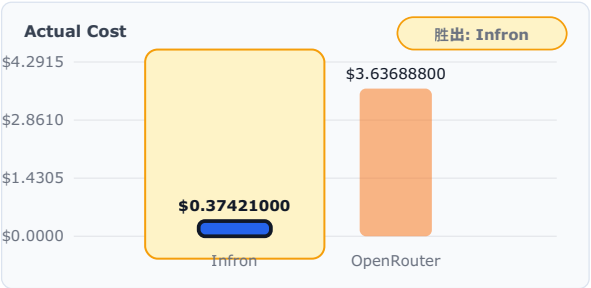
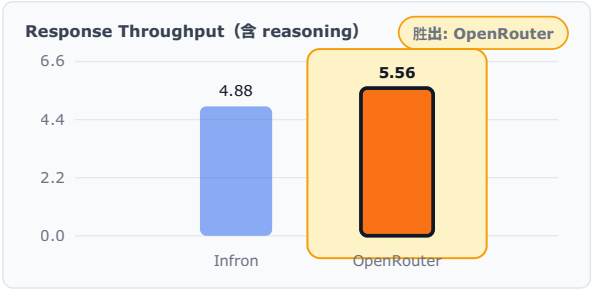
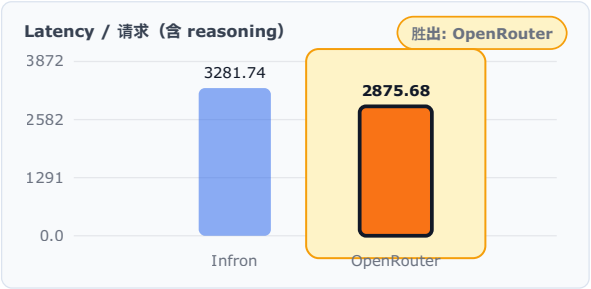


图 12：TTFT First 路由模式下的核心指标对比。柱状图同时呈现缓存、成本、吞吐、TTFT 和 latency 表现。

TTFT First 路由模式：综合雷达图

所有轴都归一化为“越外圈越好”：成本、时延、TTFT 已做反向评分。

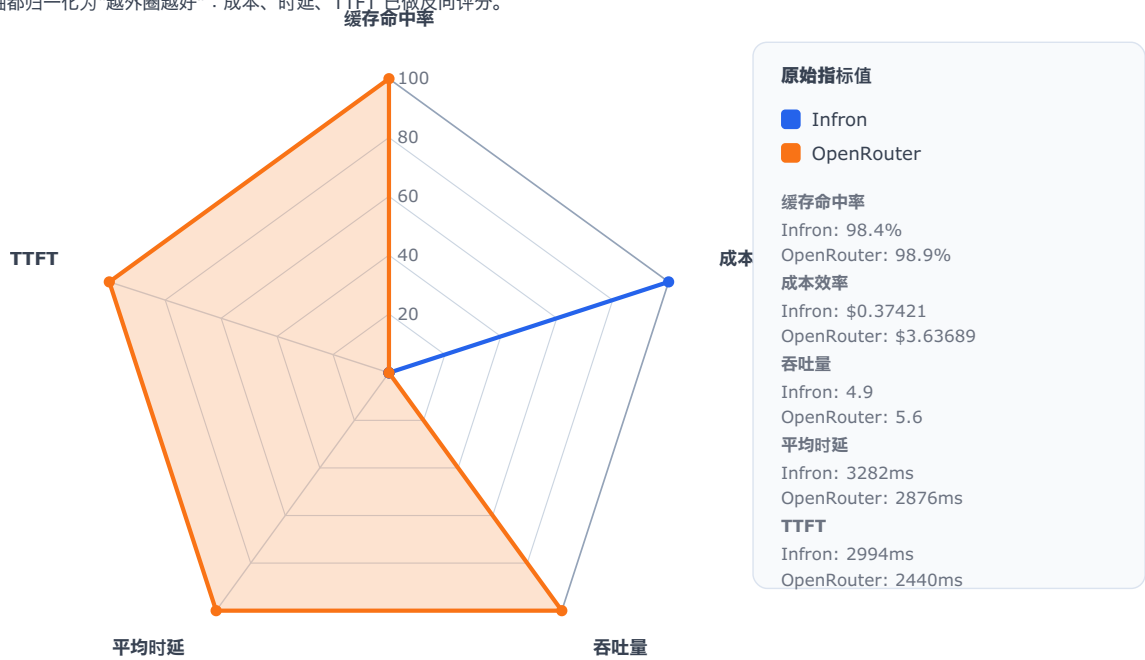


图 13：TTFT First 路由模式下的综合雷达图。所有轴都按“越外圈越好”归一化，便于快速比较两家平台的综合形状。

TTFT First 路由模式：指标生成过程对比曲线

按 group/round 顺序绘制每轮指标；曲线展示汇总均值背后的波动过程。

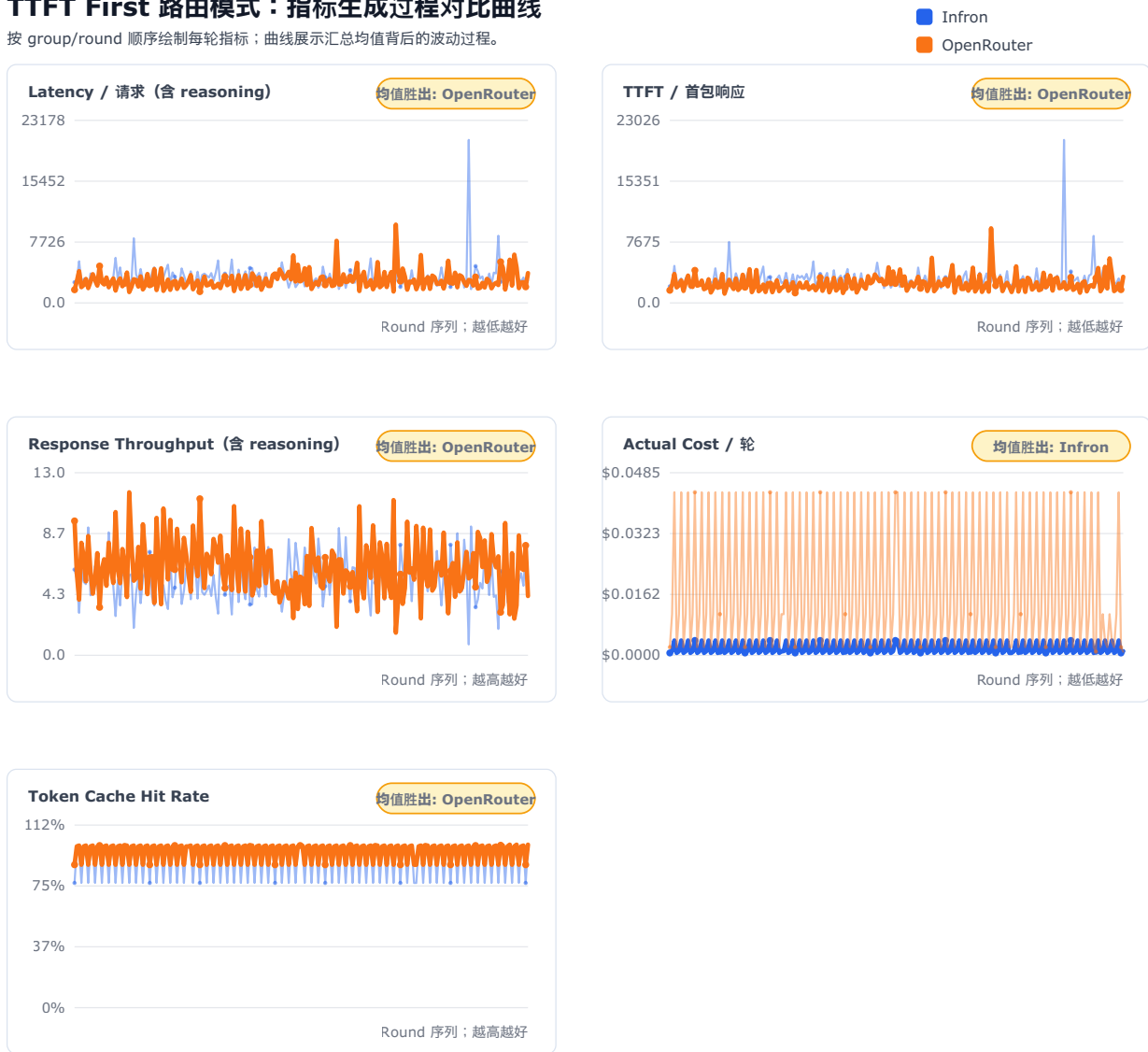


图 14: TTFT First 路由模式下的指标生成过程曲线。该图用于观察指标随 group/round 的变化，而不只依赖均值。

说明: TTFT First 中, Infron 使用 `provider.sort=ttft`; OpenRouter 使用 `provider.sort=latency` 作为可支持的对照策略。

6. Infron 技术架构与缓存/成本机制解释

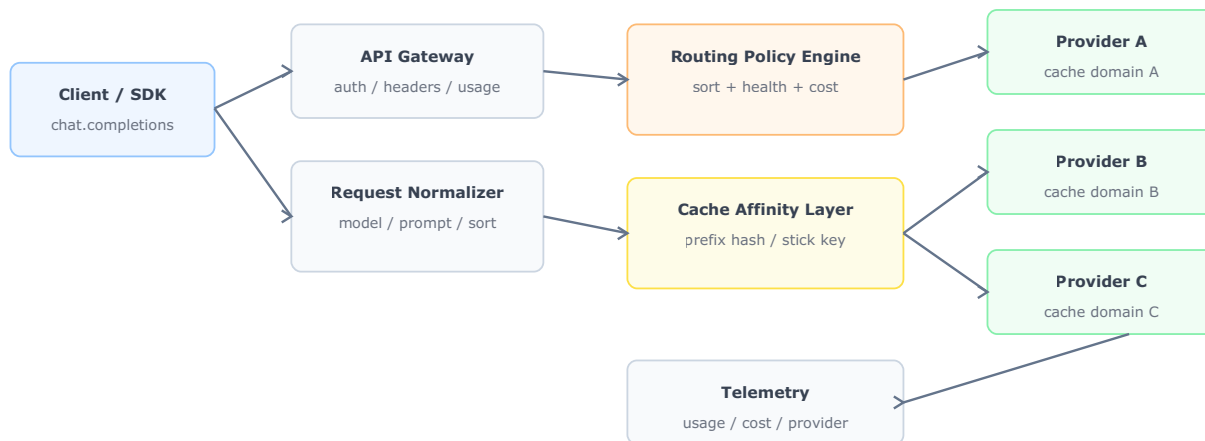
本节使用本次 benchmark 的可观测结果解释 Infron 在高 cache rate 与成本控制上的工程路径。需要说明的是，报告没有采集 Infron 内部私有 routing trace；因此下文把响应中真实返回的 provider 分布、cache read tokens、cost breakdown 和 latency/throughput 指标作为证据，用架构图解释这些结果背后的合理机制。

6.1 多 provider 路由与可观测控制面

Infron 对外提供 OpenAI-compatible API，对内需要在多个上游 provider、模型部署和路由策略之间做选择。对 prompt caching 工作负载而言，路由层不只是选择一个可用 provider，还需要同时考虑缓存亲和性、健康状态、成本、吞吐和时延目标。

Infron 多 Provider 路由与缓存控制面

OpenAI-compatible API 将请求规范化后，在健康、成本、吞吐、时延和缓存亲和性之间做路由决策。



核心点：缓存状态通常位于具体 provider/cache domain 内；路由稳定性会直接影响 cache read tokens。

图 12：Infron 多 provider 路由与缓存控制面。该图用于说明请求从统一 API 入口进入后，路由控制面如何在健康状态、策略目标、provider 选择和缓存域之间形成决策链路。

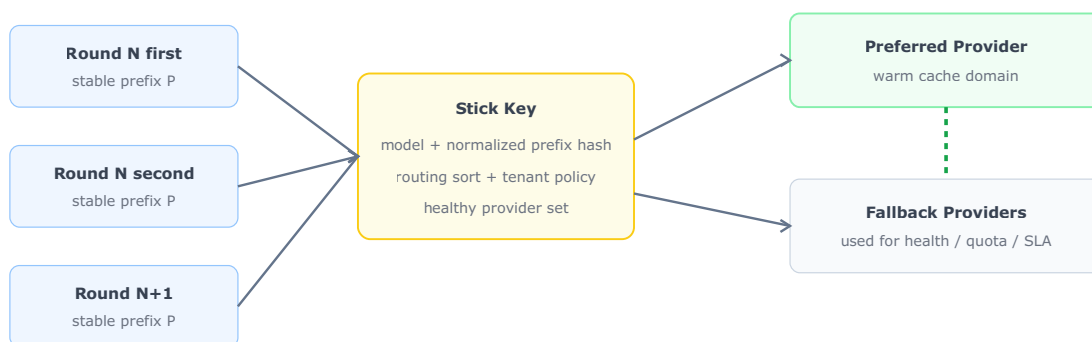
本次实验中，Infron 在不同 routing sort 下呈现出可观测的 provider 分布：throughput 主要路由到 minimax (100.00%)；price 主要路由到 minimax (100.00%)；latency 主要路由到 minimax (100.00%)；ttft 主要路由到 minimax (100.00%)。这种模式说明路由结果不是完全随机扩散，而是围绕路由目标形成了较稳定的 provider 选择。稳定的 provider 选择是高缓存命中率的前提，因为 prompt cache 通常与具体 provider、模型部署或缓存域绑定。

6.2 Provider Stick 与 Cache Affinity

Provider stick 是多 provider 网关中的缓存亲和策略：当请求具有相同或高度稳定的 prompt prefix 时，路由层倾向于把同一类请求送往同一个健康 provider 或缓存域，以减少缓存碎片化。它不等于固定永不切换 provider；当上游不可用、限流或 SLA 风险升高时，路由仍应回退到其他健康路径。

Provider Stick / Cache Affinity 机制

相同 stable prefix 的连续请求优先落入同一健康缓存域，减少跨 provider 暖缓存。



Provider stick 不等于禁用 fallback；它是在健康 provider 集合内优先

结果信号：同一 sort 内 provider 分布越集中，第二次请求越容易读取同一缓存域中的 prefix cache。

图 13: Provider stick 与 cache affinity 机制。该图表达的是工程机制假设：同类请求在健康 provider 集合内保持缓存亲和，可减少跨 provider/cache domain 的缓存碎片。

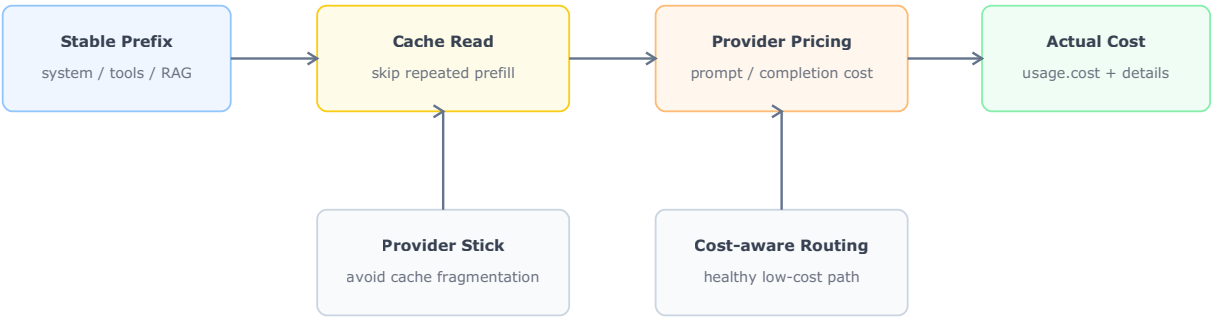
本次实验中，Infron 的缓存命中优势并非在所有 routing sort 下都成立：throughput Infron；price Infron；latency OpenRouter；ttft OpenRouter。这说明 provider stick/cache affinity 的收益依赖具体路由目标和最终上游路径。对于相同 stable prefix 的连续双请求，若 first/second 请求稳定落在同一缓存域，第二次请求更容易读取第一次请求写入或刷新后的 KV/cache 状态；若请求跨 provider、跨部署或落到不充分支持该缓存口径的路径，同样的 prompt 也可能需要分别暖缓存，从而降低整体 cache read tokens。

6.3 成本控制路径

成本控制来自三层叠加：第一层是缓存命中降低重复 prefill 的有效处理成本；第二层是 provider routing 在健康 provider 集合内选择更合适的成本路径；第三层是输出 token 与 reasoning token 对总成本的影响。本次实验中，实际成本胜出方为：throughput Infron；price Infron；latency Infron；ttft Infron。这说明缓存亲和、provider 单价、输出 token 数和 reasoning 执行情况会共同影响单位请求成本，不能只用 cache hit rate 单独解释成本结果。

Infron 成本控制路径

成本来自 token 处理、缓存读写和上游 provider 价格；缓存亲和与路由选择共同降低单位请求成本。



本次实验信号：Infron 的 Token 级缓存命中率与实际成本在不同路由模式下呈现差异化优势。
解释：高 cache read tokens 降低重复 prefill 成本；provider stick 维持缓存域稳定；成本感知 routing 在健康 provider 中选择更合适的价格路径。

图 14：Infron 成本控制路径。该图把缓存命中、provider stick、成本感知 routing 和响应 cost breakdown 连接起来，用于解释为什么 cache rate 与实际成本会同步改善。

表 7：Infron 缓存与成本控制机制的可观测证据。

机制	对 cache rate 的影响	对成本的影响	本次实验中的可观测信号
Stable prefix 识别	相同前缀更容易命中已有 cache	降低重复 prefill 的边际成本	同一 payload SHA256、第二次请求 cache read tokens 高
Provider stick / cache affinity	降低跨 provider/cache domain 的缓存碎片	减少重复暖缓存	Infron 在 sort 内 provider 分布更集中，Token 命中率更高
健康检查与 fallback	保护可用性，避免单 provider 故障	fallback 可能牺牲部分缓存收益，但降低失败成本	HTTP 状态均为 200，provider 分布仍保留少量切换可能
成本感知 routing	在满足健康和策略约束下偏向低成本路径	降低总成本和每轮成本	本轮不同 routing sort 下成本胜出方不一致，需要结合 provider 分布、cache read tokens 和 reasoning tokens 解释

因此，本轮数据更准确的结论是：高 cache rate 的关键在于路由层、缓存域和 provider 选择之间是否形成稳定亲和，而不是平台名称本身。对于长 system prompt、RAG 固定前缀、工具说明和高频模板化请求，只有当路由目标与缓存亲和一致时，这种亲和性才会转化为更高的 cache read tokens，并进一步影响单位请求成本。

7. Provider/Route 下钻分析

说明：本轮 streaming 响应已采集到部分上游 provider 标识、响应 model/id、request_id、provider cost breakdown 候选字段；下钻分析结合这些真实返回字段与可观测 telemetry（缓存命中、实际成本、latency、TTFT、throughput）解释 Infron 与 OpenRouter 内部路由差异。

表 8：Provider/Route 下钻指标。该表把 provider 分布、成本、吞吐、TTFT 和 latency 放在同一层级，用于分析路由选择如何影响最终结果。

路由偏好	平台	有效轮次	Input Tokens	Token 命中率	实际成本	成本/1K Input	响应 Throughput (含 reasoning)	TTFT	Latency/请求 (含 reasoning)	可观测路由画像
throughput	Infron	200	6207102	97.41%	\$0.36936000	\$0.000060	2.67 response tok/s	5786.47 ms	5980.69 ms	缓存亲和度高，成本控制更强
throughput	OpenRouter	200	6207808	93.54%	\$3.59489664	\$0.000579	3.39 response tok/s	4423.83 ms	4723.73 ms	速度路径更激进，优先低时延/高吞吐
price	Infron	200	6207102	98.58%	\$0.35053300	\$0.000056	2.60 response tok/s	5959.01 ms	6150.60 ms	缓存、成本、速度指标同时占优
price	OpenRouter	200	6207808	66.56%	\$0.72619654	\$0.000117	1.79 response tok/s	8499.25 ms	8930.26 ms	表现均衡但无单项极值
latency	Infron	200	6207008	98.43%	\$0.39538500	\$0.000064	5.30 response tok/s	2837.35 ms	3016.90 ms	表现均衡但无单项极值
latency	OpenRouter	200	6207808	98.90%	\$3.53757760	\$0.000570	5.49 response tok/s	2564.25 ms	2916.05 ms	速度路径更激进，优先低时延/高吞吐
ttft	Infron	200	6207008	98.43%	\$0.37421000	\$0.000060	4.88 response tok/s	2994.02 ms	3281.74 ms	表现均衡但无单项极值

路由偏好	平台	有效轮次	Input Tokens	Token 命中率	实际成本	成本/1K Input	响应 Throughput (含 reasoning)	TTFT	Latency/请求 (含 reasoning)	可观测路由画像
ttft	OpenRouter	200	6207808	98.88%	\$3.63688800	\$0.000586	5.56 response tok/s	2440.43 ms	2875.68 ms	速度路径更激进, 优先低时延/高吞吐

上游 Provider 分布

表 9：上游 provider 归因覆盖率总览。总请求数 是 first/second 请求级计数；已归因请求数 表示响应中可提取到 provider 标识的请求数。

路由偏好	平台	总请求数	已归因请求数	归因覆盖率	Provider 分布	Cost breakdown 请求数
throughput	Infron	400	400	100.00%	minimax: 400 (100.00%)	400
throughput	OpenRouter	400	400	100.00%	Groq: 374 (93.50%), SambaNova: 26 (6.50%)	400
price	Infron	400	400	100.00%	minimax: 400 (100.00%)	400
price	OpenRouter	400	400	100.00%	DeepInfra: 208 (52.00%), Mara: 192 (48.00%)	400
latency	Infron	400	400	100.00%	minimax: 400 (100.00%)	400
latency	OpenRouter	400	400	100.00%	Groq: 384 (96.00%), DeepInfra: 16 (4.00%)	400
ttft	Infron	400	400	100.00%	minimax: 400 (100.00%)	400
ttft	OpenRouter	400	400	100.00%	Groq: 392 (98.00%), DeepInfra: 8 (2.00%)	400

表 10：上游 provider 明细分布。该表按 provider 拆分请求占比、first/second 分布、覆盖轮次、时延、TTFT、token、cache 和成本，用于定位最终 A/B 差异来自哪个上游路径。

路由偏好	平台	上游 Provider	请求数	占比	first/second	覆盖轮次	Avg TTFT	Avg Latency	Prompt Tokens	Completion Tokens	Reasoning Tokens
throughput	Infron	minimax	400	100.00%	200/200	200	5786.47 ms	5980.69 ms	6207102	6400	5643
throughput	OpenRouter	Groq	374	93.50%	186/188	194	4307.80 ms	4617.41 ms	5677574	5984	7463

路由偏好	平台	上游 Provider	请求数	占比	first/second	覆盖轮次	Avg TTFT	Avg Latency	Prompt Tokens	Completion Tokens	Reasoning Tokens
throughput	OpenRouter	SambaNova	26	6.50%	14/12	20	6092.92 ms	6252.99 ms	530234	416	416
price	Infron	minimax	400	100.00%	200/200	200	5959.01 ms	6150.60 ms	6207102	6400	5635
price	OpenRouter	DeepInfra	208	52.00%	77/131	131	7934.55 ms	8497.51 ms	3289418	3328	3943
price	OpenRouter	Mara	192	48.00%	123/69	123	9111.01 ms	9399.08 ms	2918390	3072	3072
latency	Infron	minimax	400	100.00%	200/200	200	2837.35 ms	3016.90 ms	6207008	6400	6389
latency	OpenRouter	Groq	384	96.00%	192/192	192	2471.18 ms	2794.20 ms	5846912	6144	7668
latency	OpenRouter	DeepInfra	16	4.00%	8/8	8	4797.91 ms	5840.46 ms	360896	256	313
ttft	Infron	minimax	400	100.00%	200/200	200	2994.02 ms	3281.74 ms	6207008	6400	6389
ttft	OpenRouter	Groq	392	98.00%	196/196	196	2415.75 ms	2837.50 ms	6027360	6272	7800
ttft	OpenRouter	DeepInfra	8	2.00%	4/4	4	3650.17 ms	4746.30 ms	180448	128	160

7.1 缓存命中率与实际成本反向表现下钻

本节专门解释两个问题：第一，为什么某些路由模式下 Infron 的缓存命中率低于 OpenRouter；第二，为什么某些路由模式下 Infron 的实际成本高于 OpenRouter。分析只使用本轮响应中可观测的 telemetry：provider 分布、cache read tokens、usage cost、completion tokens、reasoning tokens、TTFT 与端到端 E2E 时延。

表 10-A：按路由模式拆解缓存与成本差异。缓存差值为 Infron 减 OpenRouter，成本倍数为 Infron 实际成本 / OpenRouter 实际成本。

路由偏好	缓存命中差值	Infron 成本倍数	Infron 主要上游路径	OpenRouter 主要上游路径	Reasoning Tokens 差异	主要归因
throughput	+3.88 pp	0.10x	minimax 100.0%	Groq 93.5% SambaNova 6.5%	-2236	缓存亲和、reasoning 控制和 provider 成本路径共同改善
price	+32.02 pp	0.48x	minimax 100.0%	DeepInfra 52.0% Mara 48.0%	-1380	缓存亲和、reasoning 控制和 provider 成本路径共同改善

路由偏好	缓存命中差值	Infron 成本倍数	Infron 主要上游路径	OpenRouter 主要上游路径	Reasoning Tokens 差异	主要归因
latency	-0.47 pp	0.11x	minimax 100.0%	Groq 96.0% DeepInfra 4.0%	-1592	双方主要差异来自 provider 选择和缓存域组合
ttft	-0.45 pp	0.10x	minimax 100.0%	Groq 98.0% DeepInfra 2.0%	-1571	双方主要差异来自 provider 选择和缓存域组合

分模式解释：

- throughput：Infron Token 级缓存命中率为 97.41%，OpenRouter 为 93.54%，差值 +3.88 pp；Infron 成本为 \$0.36936000，OpenRouter 为 \$3.59489664，成本倍数 0.10x。Infron 主要路径为 minimax 100.0%；OpenRouter 主要路径为 Groq 93.5%，SambaNova 6.5%。OpenRouter 比 Infron 多 2236 个 reasoning tokens。该模式下 Infron 同时取得更高缓存命中率和更低实际成本，说明当前 provider 路径与缓存域匹配较好；若速度指标未同步胜出，差异更可能来自上游响应路径和排队行为。
- price：Infron Token 级缓存命中率为 98.58%，OpenRouter 为 66.56%，差值 +32.02 pp；Infron 成本为 \$0.35053300，OpenRouter 为 \$0.72619654，成本倍数 0.48x。Infron 主要路径为 minimax 100.0%；OpenRouter 主要路径为 DeepInfra 52.0%，Mara 48.0%。OpenRouter 比 Infron 多 1380 个 reasoning tokens。该模式下 Infron 同时取得更高缓存命中率和更低实际成本，说明当前 provider 路径与缓存域匹配较好；若速度指标未同步胜出，差异更可能来自上游响应路径和排队行为。
- latency：Infron Token 级缓存命中率为 98.43%，OpenRouter 为 98.90%，差值 -0.47 pp；Infron 成本为 \$0.39538500，OpenRouter 为 \$3.53757760，成本倍数 0.11x。Infron 主要路径为 minimax 100.0%；OpenRouter 主要路径为 Groq 96.0%，DeepInfra 4.0%。OpenRouter 比 Infron 多 1592 个 reasoning tokens。该模式下 Infron 缓存命中率更低但实际成本更低，说明 provider 单价路径或输出规模对成本的影响超过了缓存差异。
- ttft：Infron Token 级缓存命中率为 98.43%，OpenRouter 为 98.88%，差值 -0.45 pp；Infron 成本为 \$0.37421000，OpenRouter 为 \$3.63688800，成本倍数 0.10x。Infron 主要路径为 minimax 100.0%；OpenRouter 主要路径为 Groq 98.0%，DeepInfra 2.0%。OpenRouter 比 Infron 多 1571 个 reasoning tokens。该模式下 Infron 缓存命中率更低但实际成本更低，说明 provider 单价路径或输出规模对成本的影响超过了缓存差异。

总体看，本轮真正影响缓存与成本的不是单一平台标签，而是“路由目标 → 实际 provider → 缓存域 → reasoning 执行 → usage/cost 返回”的链路组合。Infron 在 throughput、price 的 Token 级缓存命中率高于 OpenRouter，在 throughput、price、latency、ttft 的实际成本低于 OpenRouter。Reasoning tokens 差异主要出现在 throughput -2236；price -1380；latency -1592；ttft -1571。

- throughput 路由下：缓存命中 Infron 更优，成本 Infron 更低，throughput OpenRouter 更高，latency OpenRouter 更低，TTFT OpenRouter 更低。这说明 OpenRouter 在该路由下更偏首包与完整响应速度路径。
- price 路由下：缓存命中 Infron 更优，成本 Infron 更低，throughput Infron 更高，latency Infron 更低，TTFT Infron 更低。这说明 Infron 在该路由下更偏缓存亲和、成本控制和低时延的综合路径，OpenRouter 主要保留吞吐优势。

- `latency` 路由下：缓存命中 OpenRouter 更优，成本 Infron 更低，throughput OpenRouter 更高，latency OpenRouter 更低，TTFT OpenRouter 更低。这说明 OpenRouter 在该路由下更偏首包与完整响应速度路径。
- `ttft` 路由下：缓存命中 OpenRouter 更优，成本 Infron 更低，throughput OpenRouter 更高，latency OpenRouter 更低，TTFT OpenRouter 更低。这说明 OpenRouter 在该路由下更偏首包与完整响应速度路径。
- 脚本已支持在后续实验中采集上游 provider 标识候选字段、routing trace 候选字段、provider cost breakdown 候选字段，并可通过 `--stream` 记录 TTFT、首内容 token 与首 reasoning token 时间。当前报告只展示响应中真实存在的字段，不伪造 provider identity。

8. 分层结果：按 Prompt 长度的缓存表现

本节按 prompt 长度 tier 聚合第二次请求的 cache read tokens、Token 级缓存命中率、实际成本和时延。加粗单元表示同一长度 tier 下表现更优的一方；缓存命中率越高越好，成本、latency 和 TTFT 越低越好。

表 11: Prompt 长度分层下的总体缓存表现。

Prompt 长度 tier	目标 tokens	平台	轮数	第二次 Prompt Tokens	第二次 Cache Read Tokens	Token 级命中率	实际成本	平均 Latency	平均 TTFT
<code>short</code>	1500	Infron	268	469298	364687	77.71%	\$0.13344700	3632.64 ms	3421.09 ms
<code>short</code>	1500	OpenRouter	268	469804	381568	81.22%	\$0.46397534	3568.51 ms	3223.26 ms
<code>medium</code>	8000	Infron	268	2427854	2401389	98.91%	\$0.29008400	4367.24 ms	4173.93 ms
<code>medium</code>	8000	OpenRouter	268	2428348	2124448	87.49%	\$2.26869193	4592.34 ms	4219.52 ms
<code>long</code>	32000	Infron	264	9516956	9426250	99.05%	\$1.06595700	5840.97 ms	5605.71 ms
<code>long</code>	32000	OpenRouter	264	9517464	8602336	90.38%	\$8.76289151	6447.10 ms	6026.10 ms

表 11-2: Prompt 长度 × 路由模式的 Token 级缓存命中率。

Prompt 长度 tier	路由偏好	Infron	OpenRouter	胜出方
<code>short</code>	<code>throughput</code>	78.78%	86.31%	OpenRouter
<code>short</code>	<code>price</code>	79.11%	63.32%	Infron
<code>short</code>	<code>latency</code>	76.47%	87.62%	OpenRouter
<code>short</code>	<code>ttft</code>	76.47%	87.62%	OpenRouter
<code>medium</code>	<code>throughput</code>	98.96%	97.03%	Infron
<code>medium</code>	<code>price</code>	98.97%	55.06%	Infron

Prompt 长度 tier	路由偏好	Infron	OpenRouter	胜出方
medium	latency	98.85%	98.94%	OpenRouter
medium	ttft	98.85%	98.91%	OpenRouter
long	throughput	97.94%	93.00%	Infron
long	price	99.44%	69.66%	Infron
long	latency	99.41%	99.45%	OpenRouter
long	ttft	99.41%	99.43%	OpenRouter

9. 分层结果：按实验组的稳定性检查

throughput

表 12-1: throughput 路由模式下的 group-level 稳定性检查。

平台	组别	轮数	成功轮数	Token 级命中率	实际成本
Infron	1	50	50	93.72%	\$0.11228200
Infron	2	50	50	98.56%	\$0.08750700
Infron	3	50	50	98.65%	\$0.08611600
Infron	4	50	50	98.63%	\$0.08345500
OpenRouter	1	50	50	94.13%	\$0.91568400
OpenRouter	2	50	50	94.39%	\$0.95684160
OpenRouter	3	50	50	98.87%	\$0.94808160
OpenRouter	4	50	50	86.53%	\$0.77428944

price

表 12-2: price 路由模式下的 group-level 稳定性检查。

平台	组别	轮数	成功轮数	Token 级命中率	实际成本
Infron	1	50	50	98.53%	\$0.09330200
Infron	2	50	50	98.57%	\$0.08736200
Infron	3	50	50	98.74%	\$0.08599400
Infron	4	50	50	98.46%	\$0.08387500
OpenRouter	1	50	50	79.03%	\$0.18039918
OpenRouter	2	50	50	72.88%	\$0.16937106
OpenRouter	3	50	50	44.54%	\$0.20541523
OpenRouter	4	50	50	70.30%	\$0.17101107

latency

表 12-3： latency 路由模式下的 group-level 稳定性检查。

平台	组别	轮数	成功轮数	Token 级命中率	实际成本
Infron	1	50	50	98.40%	\$0.11318500
Infron	2	50	50	98.49%	\$0.09532600
Infron	3	50	50	98.44%	\$0.09486400
Infron	4	50	50	98.40%	\$0.09201000
OpenRouter	1	50	50	98.85%	\$0.91566480
OpenRouter	2	50	50	98.90%	\$0.95682240
OpenRouter	3	50	50	98.87%	\$0.94805280
OpenRouter	4	50	50	98.99%	\$0.71703760

ttft

表 12-4： ttft 路由模式下的 group-level 稳定性检查。

平台	组别	轮数	成功轮数	Token 级命中率	实际成本
Infron	1	50	50	98.40%	\$0.09201000
Infron	2	50	50	98.49%	\$0.09532600
Infron	3	50	50	98.44%	\$0.09486400
Infron	4	50	50	98.40%	\$0.09201000
OpenRouter	1	50	50	98.85%	\$0.91566480
OpenRouter	2	50	50	98.90%	\$0.95682240
OpenRouter	3	50	50	98.87%	\$0.94805280
OpenRouter	4	50	50	98.92%	\$0.81634800

10. 讨论：业务价值、适用边界与工程启示

四种 routing sort 对应不同业务目标，需要结合缓存、成本、吞吐、端到端时延和 TTFT 一起判断。
throughput 更适合批处理、异步生成、长文本生产和离线任务； price 更适合高频低毛利调用、固定模板请求、客服/营销自动化等成本敏感场景； latency 更适合交互式产品、Agent 工具调用链、实时辅助写作和用户等待成本较高的场景； ttft 更适合首包体验敏感、需要快速给用户反馈的流式交互场景。

路由模式	主要业务目标	本轮数据体现	适用场景	注意事项
throughput	最大化单位时间输出能力	缓存 Infron 占优，成本 Infron 占优， throughput OpenRouter 占优， latency OpenRouter 占优， TTFT OpenRouter 占优	批量内容生成、离线摘要、后台数据加工	适合吞吐优先任务，但需接受缓存和成本可能被速度目标牺牲

路由模式	主要业务目标	本轮数据体现	适用场景	注意事项
price	最小化单位请求和单位 token 成本	缓存 Infron 占优，成本 Infron 占优，throughput Infron 占优，latency Infron 占优，TTFT Infron 占优	高频模板化请求、客服自动化、营销触达、RAG 固定前缀	更适合成本和体验受控的在线业务，但吞吐可能不是最优
latency	最小化用户可感知等待时间	缓存 OpenRouter 占优，成本 Infron 占优，throughput OpenRouter 占优，latency OpenRouter 占优，TTFT OpenRouter 占优	在线聊天、Agent 调用链、IDE/写作辅助、实时运营工具	适合交互式任务，但同时约束缓存命中和单位成本
ttft	最小化流式首包响应时间	缓存 OpenRouter 占优，成本 Infron 占优，throughput OpenRouter 占优，latency OpenRouter 占优，TTFT OpenRouter 占优	流式聊天、实时 Copilot、首屏反馈、长思考任务的进度感知	适合首包体验优先任务，但仍需检查完整响应时延和吞吐是否满足 SLA

从业务决策角度看，prompt caching 的价值不只体现在单次请求省钱，而是体现在大规模重复上下文请求的边际成本下降。若业务请求结构高度模板化，应优先关注 Token 级命中率和实际成本；若业务以用户实时体验为核心，应同时约束 latency；若业务为后台批量生成，则 throughput 可能比单请求 latency 更重要。

因此，本实验的推荐读法是：先确认 Input Tokens 是否完全可比，再按业务目标选择主指标，最后检查其他指标是否出现不可接受的副作用。例如某个平台吞吐更高但缓存命中显著较低，可能适合批处理，却未必适合需要稳定成本结构的高频在线业务。

11. 结论

表 13：路由模式级结论快照。该表综合缓存命中、成本、throughput、latency 和 TTFT，避免只按单一指标排序。

路由偏好	缓存命中更优	成本更低	Throughput 更高	Latency 更低	TTFT 更低	综合解读
throughput	Infron	Infron	OpenRouter	OpenRouter	OpenRouter	OpenRouter 综合占优（3/5 可比指标）
price	Infron	Infron	Infron	Infron	Infron	Infron 综合占优（5/5 可比指标）
latency	OpenRouter	Infron	OpenRouter	OpenRouter	OpenRouter	OpenRouter 综合占优（4/5 可比指标）
ttft	OpenRouter	Infron	OpenRouter	OpenRouter	OpenRouter	OpenRouter 综合占优（4/5 可比指标）

12. 局限性、缺失数据与后续实验计划

本报告区分“已观测事实”和“机制解释”。已观测事实来自响应 usage、cost、latency、TTFT、cache tokens、provider 字段和导出的请求级 telemetry；机制解释用于说明这些结果背后的合理工程路径，不代表平台内部私有实现的直接证据。

表 14：当前报告的局限性与后续补充计划。

缺失或不足	对结论的影响	后续补充方式	当前处理方式
上游完整 routing trace	无法逐跳证明每次请求的 provider 选择、fallback 和重试路径	待补充：provider routing trace / decision log / fallback reason	仅使用响应中真实返回的 provider 字段和 provider 分布做归因
Provider cost breakdown 全量字段	无法进一步拆分平台费、provider 费、cache read/write 成本	待补充：provider cost breakdown 明细、缓存读写计费项	只统计响应明确返回的 cost/cost_details
显著性检验	已补充 bootstrap 95% CI 与 paired sign-flip permutation test；尚未给出 standardized effect size	待补充：Cohen's d / Cliff's delta 等 effect size	使用严格 A/B 配对和 input token 相等过滤降低混杂偏差
P95/P99 latency	已补充 P50/P95/P99 latency 与 TTFT；尚未计算 IQR 和 tail amplification	待补充：IQR、max、tail amplification ratio	当前展示均值、P50/P95/P99 和过程曲线
多模型泛化	单模型实验不能直接外推到所有模型	待补充：DeepSeek、Qwen、Claude、GPT 系列跨模型实验	结论限于 minimax/minimax-m2.7 本轮样本
真实业务语料	本轮使用内置代表性业务模板，不等于客户生产语料	待补充：脱敏真实 RAG、Agent、客服、代码生成、长文摘要业务数据集	脚本已支持 --dataset-file JSONL 输入
并发压力与长期运行	本轮使用 workers 并发执行，但不是长时间 soak test	待补充：并发阶梯压测、24h soak test、cache TTL/eviction 观测	当前解释 4*50 并发执行窗口内的 A/B 结果

后续实验可以继续采用核心 A/B 配对方法：保持 payload SHA256、usage.prompt_tokens 偏差不超过 50 tokens 的配对过滤和 request-level telemetry，同时增加 routing trace、provider cost breakdown、尾延迟分位数和业务语料分层。这样可以把本报告扩展为更完整的生产决策评估框架。

13. 可复现性附录：Benchmark 数据集

本节给出复现结论和图表所需的数据文件。配对级 CSV 是报告中所有总览表、核心指标图和结论快照的直接输入；请求级 JSONL 保留每一次 first/second 请求的原始 telemetry，便于审计 provider、usage、cost、latency、TTFT 与缓存字段。为满足单文件审计，报告后文完整嵌入本次实验使用的配对级数据、请求级数据、过滤后记录和剔除样本记录。

数据文件
export/minimax_m27_all_experiments/ routing_sort_cache_cost_ab_4x50_stream_ttft_reasoning_default_length_tiers_chat_completions_20260707/ benchmark_pairs.csv
export/minimax_m27_all_experiments/ routing_sort_cache_cost_ab_4x50_stream_ttft_reasoning_default_length_tiers_chat_completions_20260707/ benchmark_requests.jsonl

字段字典：

字段	含义
sort/group/round	A/B 配对键；同一键下 Infron 与 OpenRouter first/second 输入 token 偏差不超过 50
prompt_length_tier	Prompt 长度分层标签；未启用分层时为 default
target_prompt_tokens	该 tier 的目标 prompt token 规模；实际比较仍以响应返回的 usage.prompt_tokens 为准
*_pair_cost_usd	first + second 两次请求的真实响应成本
*_avg_latency_ms	first/second 两次请求 latency 均值
*_avg_ttft_ms	first/second 两次请求 TTFT 均值
*_response_throughput_tps	两次请求 completion tokens / 两次请求总 latency seconds
*_second_cache_read_tokens	第二次请求读取缓存的 token 数
*_second_cache_hit_rate	第二次请求 cache read tokens / 第二次请求 prompt tokens
*_provider	响应中可观测的上游 provider 标识

14. 可复现性附录：代码

14.1 完整 A/B Testing 实验脚本

完整脚本全文已嵌入同名 HTML/Markdown 完整版报告。PDF 版保留脚本文件路径、SHA256 和复现命令，避免全量源码与数据集导致 PDF 排版不可用。

文件：scripts/rerun_routing_sort_cache_cost_ab.py

SHA256：8b960eea701a34eb5d483de621b35a86004b90afbbc65ebf3c4d492e3335a339

大小：276651 bytes

完整内容见同名 HTML/Markdown 完整版报告。

14.2 A/B Testing 核心执行逻辑摘录

以下代码展示完整的 A/B testing 执行逻辑：同一 payload 分别发送到 Infron 与 OpenRouter；每轮发送 first/second 两次请求；streaming 读取到 [DONE] 前的最终 SSE JSON chunk；提取 usage、cost、cost_details、TTFT、cache tokens 和 provider 标识；最后只保留 A/B 两边 input-token 偏差不超过 50 tokens 的配对样本。

```
from __future__ import annotations

import json
import os
import time
from collections import defaultdict
from urllib.request import Request, urlopen

MODEL = 'minimax/minimax-m2.7'
SORT_MODES = ('throughput', 'price', 'latency', 'ttft')
PROVIDER_SORT_OVERRIDES = (('infron', 'ttft'): 'ttft', ('openrouter', 'ttft'): 'latency')
```

```

CACHE_PREFIX = ' '.join(['stable prompt cache prefix'] * 220)

PROVIDERS = {
    'infron': {
        'base_url': os.environ['INFRON_BASE_URL'].rstrip('/'),
        'api_key': os.environ['INFRON_API_KEY'],
        'headers': {},
    },
    'openrouter': {
        'base_url': os.environ['OPENROUTER_BASE_URL'].rstrip('/'),
        'api_key': os.environ['OPENROUTER_API_KEY'],
        'headers': {
            'HTTP-Referer': os.environ.get('OPENROUTER_HTTP_REFERER', 'https://example.com'),
            'X-Title': os.environ.get('OPENROUTER_APP_TITLE', 'cache-ab-test'),
        },
    },
}

def payload(sort_mode: str, provider_name: str) -> dict:
    return {
        'model': MODEL,
        'messages': [
            {'role': 'system', 'content': CACHE_PREFIX},
            {'role': 'user', 'content': 'Reply with exactly: cache probe ok'},
        ],
        'temperature': 0,
        'max_tokens': 16,
        'stream': True,
        'stream_options': {'include_usage': True},
        'usage': {'include': True},
        'provider': {'sort': PROVIDER_SORT_OVERRIDES.get((provider_name, sort_mode), sort_mode)},
        'allow_fallbacks': True,
    }

def read_sse(resp, started: float) -> tuple[dict, dict]:
    assembled, usage = {}, {}
    ttft_ms = first_reasoning_ms = first_content_ms = None
    chunks = 0
    for raw in resp:
        line = raw.decode('utf-8', errors='replace').strip()
        if not line.startswith('data:'):
            continue
        data = line[5:].strip()
        if data == '[DONE]':
            continue
        chunk = json.loads(data)
        now_ms = (time.monotonic() - started) * 1000
        chunks += 1
        if ttft_ms is None:
            ttft_ms = now_ms
        for key in ('id', 'model', 'provider', 'request_id', 'system_fingerprint', 'cost',
                    'cost_details', 'provider_cost_details', 'cost_breakdown'):
            if key in chunk:
                assembled[key] = chunk[key]
        if isinstance(chunk.get('usage'), dict):
            usage = chunk['usage']
        for choice in chunk.get('choices') or []:
            delta = choice.get('delta') or {}
            if first_content_ms is None and delta.get('content'):
                first_content_ms = now_ms
            if first_reasoning_ms is None and (delta.get('reasoning') or
            delta.get('reasoning_content') or delta.get('reasoning_details')):
                first_reasoning_ms = now_ms

```



```

    if usage:
        assembled['usage'] = usage
    return assembled, {'tft_ms': tft_ms, 'first_content_token_ms': first_content_ms,
'first_reasoning_token_ms': first_reasoning_ms, 'stream_chunk_count': chunks}

def cost_value(body: dict) -> float | None:
    if isinstance(body.get('cost'), (int, float)):
        return float(body['cost'])
    usage = body.get('usage') or {}
    return float(usage['cost']) if isinstance(usage.get('cost'), (int, float)) else None

def cache_read_tokens(usage: dict) -> int:
    details = usage.get('prompt_tokens_details') or usage.get('input_tokens_details') or {}
    return int(details.get('cached_tokens') or details.get('cache_read_tokens') or 0)

def reasoning_tokens(usage: dict) -> int:
    details = usage.get('completion_tokens_details') or {}
    return int(details.get('reasoning_tokens') or usage.get('reasoning_tokens') or 0)

def send(provider_name: str, sort_mode: str) -> dict:
    provider = PROVIDERS[provider_name]
    body = json.dumps(payload(sort_mode, provider_name)).encode('utf-8')
    headers = {
        'Authorization': f"Bearer {provider['api_key']}",
        'Content-Type': 'application/json',
        'Accept': 'text/event-stream',
        'Connection': 'keep-alive',
        'User-Agent': 'GrowthPulse/benchmark-reproducer',
        **provider.get('headers', {}),
    }
    started = time.monotonic()
    with urlopen(Request(provider['base_url'] + '/chat/completions', data=body, headers=headers,
method='POST'), timeout=120) as resp:
        response_body, stream = read_sse(resp, started)
    latency_ms = (time.monotonic() - started) * 1000
    usage = response_body.get('usage') or {}
    return {
        'status': 200,
        'latency_ms': round(latency_ms, 3),
        **stream,
        'usage': usage,
        'provider_name': response_body.get('provider'),
        'cost': cost_value(response_body),
        'prompt_tokens': int(usage.get('prompt_tokens') or usage.get('input_tokens') or 0),
        'completion_tokens': int(usage.get('completion_tokens') or usage.get('output_tokens') or 0),
        'reasoning_tokens': reasoning_tokens(usage),
        'cache_read_tokens': cache_read_tokens(usage),
        'cost_details': response_body.get('cost_details') or
response_body.get('provider_cost_details') or response_body.get('cost_breakdown'),
    }

def run_ab(groups=4, rounds=50) -> list[dict]:
    records = []
    for sort_mode in SORT_MODES:
        for group in range(1, groups + 1):
            for round_no in range(1, rounds + 1):
                for provider in ('infron', 'openrouter'):
                    first = send(provider, sort_mode)
                    second = send(provider, sort_mode)
                    records.append({'sort': sort_mode, 'provider': provider, 'group': group,
'round': round_no, 'first': first, 'second': second})
    return records

```

```

def input_token_tolerant_pairs(records: list[dict], tolerance: int = 50) -> list[dict]:
    by_key = defaultdict(dict)
    for item in records:
        by_key[(item['sort'], item['group'], item['round'])][item['provider']] = item
    kept = []
    for providers in by_key.values():
        a, b = providers.get('infron'), providers.get('openrouter')
        if not a or not b:
            continue
        a_tokens = (a['first']['prompt_tokens'], a['second']['prompt_tokens'])
        b_tokens = (b['first']['prompt_tokens'], b['second']['prompt_tokens'])
        if all(token > 0 for token in (*a_tokens, *b_tokens)) and all(abs(x - y) <= tolerance for x,
y in zip(a_tokens, b_tokens)):
            kept.extend([a, b])
    return kept

# records = run_ab(groups=4, rounds=50)
# filtered_records = input_token_tolerant_pairs(records)

```

14.3 离线聚合复现代码

以下代码只依赖 Python 标准库，可从配对级 CSV 复现总览指标、胜出方判断和核心图表输入数据。

```

from __future__ import annotations

import csv
from collections import defaultdict
from pathlib import Path

PAIR_CSV = Path('export/minimax_m27_all_experiments/
routing_sort_cache_cost_ab_4x50_stream_ttft_reasoning_default_length_tiers_chat_completions_20260707
/benchmark_pairs.csv')

def f(row, key):
    value = row.get(key, '')
    return float(value) if value not in {'', 'None', 'N/A'} else 0.0

rows = list(csv.DictReader(PAIR_CSV.open(newline='', encoding='utf-8')))
summary = defaultdict(lambda: defaultdict(list))
for row in rows:
    for provider in ('infron', 'openrouter'):
        summary[row['sort']][provider].append(row)

def aggregate(items, provider):
    total_latency_ms = sum(f(row, f'{provider}_first_latency_ms') + f(row, f'{provider}
_second_latency_ms') for row in items)
    completion_tokens = sum(f(row, f'{provider}_first_completion_tokens') + f(row, f'{provider}
_second_completion_tokens') for row in items)
    second_prompt_tokens = sum(f(row, f'{provider}_second_prompt_tokens') for row in items)
    second_cache_read_tokens = sum(f(row, f'{provider}_second_cache_read_tokens') for row in items)
    return {
        'rounds': len(items),
        'input_tokens': int(sum(f(row, f'{provider}_input_tokens_total') for row in items)),
        'cost_usd': sum(f(row, f'{provider}_pair_cost_usd') for row in items),
        'latency_ms': total_latency_ms / (len(items) * 2) if items else 0,
        'throughput_tps': completion_tokens / (total_latency_ms / 1000) if total_latency_ms else 0,
        'cache_hit_rate': second_cache_read_tokens / second_prompt_tokens if second_prompt_tokens
    else 0,
    }

for sort_mode, providers in summary.items():

```

```
infron = aggregate(providers['infron'], 'infron')
openrouter = aggregate(providers['openrouter'], 'openrouter')
assert infron['input_tokens'] == openrouter['input_tokens']
winners = {
    'cache': 'Infron' if infron['cache_hit_rate'] > openrouter['cache_hit_rate'] else
'OpenRouter',
    'cost': 'Infron' if infron['cost_usd'] < openrouter['cost_usd'] else 'OpenRouter',
    'throughput': 'Infron' if infron['throughput_tps'] > openrouter['throughput_tps'] else
'OpenRouter',
    'latency': 'Infron' if infron['latency_ms'] < openrouter['latency_ms'] else 'OpenRouter',
}
print(sort_mode, {'infron': infron, 'openrouter': openrouter, 'winners': winners})
```

15. 可复现性附录：100% 原始 Benchmark 数据集

100% 原始 benchmark 数据集已嵌入同名 HTML/Markdown 完整版报告。PDF 版只保留数据文件路径、大小、SHA256 与用途，避免数 MB JSONL/JSON 造成 PDF 渲染超时。

15.1 配对级 benchmark 数据集 CSV

文件： `export/minimax_m27_all_experiments/
routing_sort_cache_cost_ab_4x50_stream_ttft_reasoning_default_length_tiers_chat_completions_20260707/
benchmark_pairs.csv`

SHA256： `ea1de14890f09229c8826f9480629255b15b3331057b18407efee032fe534e85`

大小： `348863` bytes

完整内容见同名 HTML/Markdown 完整版报告。

15.2 请求级原始 benchmark 数据集 JSONL

文件： `export/minimax_m27_all_experiments/
routing_sort_cache_cost_ab_4x50_stream_ttft_reasoning_default_length_tiers_chat_completions_20260707/
benchmark_requests.jsonl`

SHA256： `a247926e7f03573a2ca39abcc9c62c872d767a0878a50f0d47d9daad0c875a73`

大小： `5087483` bytes

完整内容见同名 HTML/Markdown 完整版报告。

15.3 过滤后原始记录 records.json

文件： `export/minimax_m27_all_experiments/
routing_sort_cache_cost_ab_4x50_stream_ttft_reasoning_default_length_tiers_chat_completions_20260707/
records.json`

SHA256： `b8ee863f8b679315adcf031a3881e403bb763912b26bc43bfaa7e8939c91c84`

大小： `8317185` bytes

完整内容见同名 HTML/Markdown 完整版报告。

15.4 剔除样本审计记录 records_excluded.json

文件: `export/minimax_m27_all_experiments/
routing_sort_cache_cost_ab_4x50_stream_ttft_reasoning_default_length_tiers_chat_completions_20260707/
records_excluded.json`

SHA256: `a4e17ae4dae862aae9f36cf63d5e892e9bfb67264b917b1febd1c786ddedef7b`

大小: `19` bytes

完整内容见同名 HTML/Markdown 完整版报告。